

2025 - 2026

PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

SPÉCIALITÉ : INFORMATIQUE

ELASTIC WATCH

**Plateforme Analytique Avancée avec BI
et Prédiction Machine Learning**

Réalisé par: SbouiElaa

Encadré par: Mr Mohamed Hedi Gharbi

Encadrant ESPRIT: Mm Manel Khamassi

Encadrant Entreprise: Tac-Tic



Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I
validate the submission of the student's report:

- Nom & Prénom /Name & Surname : Sboui Elaa

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname :Mohamed Hedi.GHARBI.....

Cachet & Signature / Stamp & Signature

ELASTIC SOLUTIONS
B.P:129 Technopole El Ghazela
2088, Cité El Ghazela, Ariana, Tunisie
contact@elastic-solutions.com
Tél: 36 36 55 58

Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname :Khamassi Manel.....

Signature / Signature



Ce formulaire doit être rempli, signé et **scanné**/This form must be completed, signed and **scanned**.

Ce formulaire doit être introduit après la page de garde/This form must be inserted after the cover page.

Dédicaces

*À mes parents,
Chaker et Mabrouka,*

Je vous dédie ce travail du fond du cœur. Vous n'avez jamais arrêté de croire en moi. Vos sacrifices silencieux et votre soutien sans limites m'ont accompagné à chaque étape de ma vie. Aucun mot ne sera jamais assez fort pour vous dire tout l'amour, le respect et la reconnaissance que j'ai pour vous.

*À mes frères et sœurs,
Merci pour votre amour, vos encouragements et d'avoir toujours été présents, que ce soit dans les moments de doute ou dans les moments de bonheur.*

*À tous mes enseignants de l'ESPRIT,
Merci de m'avoir transmis non seulement des connaissances, mais surtout la passion du métier d'ingénieur et l'envie de toujours bien faire.*

*À mes amis,
avec qui j'ai partagé les longues nuits de travail, les difficultés et les petites victoires. On a tout traversé ensemble.
Un merci tout particulier à Emna, ma complice de toutes ces années. Merci pour tous les bons moments et cette belle entente qu'on a toujours eue.*

Enfin, je pense à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce projet. Ce travail vous est aussi dédié, avec toute ma gratitude et mon affection.

Elaa SBOUI

Remerciements

Ce travail n'aurait pu voir le jour sans la contribution précieuse de nombreuses personnes, à qui j'adresse ma plus profonde reconnaissance.

Mes premiers remerciements vont à **M. Mohamed Hedi GHARBI**, encadrant entreprise chez *TAC-TIC*, pour sa disponibilité remarquable, sa vision technique claire et la confiance qu'il m'a accordée tout au long de ce stage. Ses conseils avisés m'ont permis de dépasser les obstacles et d'élever la qualité de ce projet à un niveau bien supérieur à mes attentes initiales.

Je tiens également à exprimer ma gratitude à **Mme Manel KHAMASSI**, encadrante académique à l'ESPRIT, pour son suivi rigoureux, ses orientations méthodologiques et sa disponibilité tout au long de cette année. Son accompagnement a été précieux pour la structuration de ce rapport.

Ma gratitude s'étend à l'ensemble de l'équipe de *TAC-TIC*, dont l'accueil chaleureux et le professionnalisme ont rendu cette expérience aussi enrichissante que formatrice. Intégrer une entreprise spécialisée dans les télécommunications et les nouvelles technologies m'a offert une fenêtre concrète sur les défis réels du secteur.

Enfin, je remercie chaleureusement ma famille et mes proches pour leur soutien moral inébranlable, particulièrement dans les moments de doute.

Elaa SBOUI — Tunis, 2026

Table des matières

Dédicaces	i
Remerciements	ii
Liste des figures	vii
Liste des tableaux	ix
Liste des abréviations	xi
Introduction Générale	1
1 Contexte général du projet	2
1.1 Introduction	2
1.2 Cadre général du stage	2
1.3 Présentation de l'entreprise d'accueil — TAC-TIC	3
1.3.1 Présentation de TAC-TIC	3
1.3.2 Services proposés par TAC-TIC	3
1.4 Présentation du projet Elastic Watch	4
1.4.1 Description générale	4
1.4.2 Problématique	4
1.5 Étude de l'existant	4
1.5.1 Panorama des solutions existantes	4
1.5.2 Tableau comparatif global	6
1.5.3 Solution proposée — Elastic Watch	6
1.6 Méthodologie de travail	7
1.6.1 Choix du methode Scrum	7
1.6.2 Les rôles dans la méthodologie Agile	8
1.6.3 Product Backlog	9
1.6.4 Planification des sprints	12
1.6.5 Sprint Backlog	13
1.7 Conclusion	15

2	Sprint 0 — Analyse et Spécification des Besoins	16
2.1	Introduction	16
2.2	Identification des acteurs	16
2.3	Spécification des besoins	16
2.3.1	Besoins fonctionnels	16
2.3.2	Besoins non fonctionnels	17
2.4	Modélisation UML	17
2.4.1	Diagramme des cas d'utilisation global	17
2.4.2	Diagramme de classes	19
2.5	Environnement de travail	20
2.5.1	Frameworks et langages utilisés	20
2.5.2	Environnement logiciel	20
2.6	Architecture du système	24
2.6.1	Architecture logique	24
2.6.2	Architecture physique	24
2.6.3	Data Warehouse — schéma en étoile	25
2.7	Pilotage du projet avec Scrum	26
2.7.1	Backlog produit	26
2.7.2	Sprint Backlog	27
2.8	Sprint 0 — Authentification et gestion des espaces	27
2.8.1	Analyse des besoins	27
2.8.2	Conception	28
2.8.3	Réalisation	31
2.8.4	Tests du sprint	34
2.9	Conclusion	34
3	Sprint 1 — Gestion des Passerelles et des Capteurs	35
3.1	Introduction	35
3.2	Analyse des besoins	35
3.3	Conception	36
3.3.1	Types de passerelles supportées	36
3.3.2	Types de capteurs supportés	37
3.4	Réalisation	37
3.5	Tests du sprint	38
3.6	Conclusion	38
4	Sprint 2 — Ingestion IoT et Tableau de Bord Décisionnel	39
4.1	Introduction	39
4.2	Analyse des besoins	39
4.3	Conception	40
4.3.1	Point d'entrée d'ingestion IoT	40
4.3.2	Points d'entrée du tableau de bord	41

4.3.3	Alimentation du Data Warehouse	41
4.4	Réalisation	42
4.5	Tests du sprint	42
4.6	Conclusion	43
5	Sprint 3 — Intelligence Artificielle et Apprentissage Automatique	44
5.1	Introduction	44
5.2	Analyse des besoins	44
5.2.1	Justification du choix des modèles	45
5.3	Conception de l'architecture d'apprentissage	45
5.4	Modèle Prophet — Prévission à long terme	45
5.5	Modèle Isolation Forest — Détection d'anomalies	48
5.6	Comparaison des modèles	49
5.7	Conclusion	49
6	Sprint 4 — Système d'Alertes et Supervision	50
6.1	Introduction	50
6.2	Analyse des besoins	50
6.3	Conception	52
6.4	Réalisation	53
6.5	Tests du sprint	55
6.6	Conclusion	55
7	Sprint 5 — Administration, Chatbot IA et Déploiement Docker	56
7.1	Introduction	56
7.2	Analyse des besoins	56
7.3	Module d'administration des utilisateurs	57
7.4	Assistant conversationnel — Llama 3.2 en local	58
7.4.1	Justification du choix d'un modèle local	60
7.5	Gestion des favoris	60
7.6	Pipeline ETL ,conteneurisation Docker et Déploiement Cloud	61
7.6.1	Pipeline ETL	61
7.6.2	Conteneurisation Docker	64
7.7	Déploiement Cloud	64
7.8	Tests du sprint	67
7.9	Conclusion	67
8	Tests et Validation	68
8.1	Introduction	68
8.2	Stratégie de tests	68
8.3	Tests back-end (PHPUnit)	69
8.4	Tests front-end (Vitest)	69
8.5	Tests d'apprentissage automatique (pytest)	69

8.5.1	Évaluation de la détection d'anomalies	69
8.6	Tests de performance	70
8.7	Tests de sécurité et exigences non fonctionnelles	70
8.8	Conclusion	71
Conclusion Générale et Perspectives		72
Bibliographie		74
Annexes		75
	Annexe A. Glossaire	75
	Annexe B. Checklist de déploiement	77
	Annexe C. Endpoints API complets	78

Liste des figures

1.1	Logo de TAC-TIC	3
1.2	La méthode Scrum	7
2.1	Diagramme des cas d'utilisation global du système Elastic Watch	18
2.2	Diagramme de classes de la couche métier d'Elastic Watch	19
2.3	Logo Render	20
2.4	Logo Railway	20
2.5	Logo Vercel	21
2.6	Logo MySQL	21
2.7	Logo Postman	21
2.8	Logo Docker	22
2.9	Logo GitHub	22
2.10	Logo PlantUML	22
2.11	Logo Visual Studio Code	23
2.12	Logo XAMPP	23
2.13	Architecture logique de la plateforme Elastic Watch	24
2.14	Architecture physique de déploiement d'Elastic Watch	25
2.15	Diagramme de cas d'utilisation du Sprint 1	28
2.16	Diagramme de séquence de l'authentification à deux facteurs (JWT + OTP)	29
2.17	Diagramme de séquence de la réinitialisation du mot de passe	30
2.18	Diagramme de séquence de la gestion hiérarchique des espaces	31
2.19	Page de connexion de la plateforme Elastic Watch	32
2.20	Formulaire de vérification du code à usage unique (OTP)	32
2.21	Formulaire de changement de mot de passe	33
2.22	Module de gestion hiérarchique des espaces	33
3.1	Diagramme de cas d'utilisation du Sprint 1	36
3.2	Module de gestion des passerelles IoT et AI	37
3.3	Module de gestion des capteurs et de leurs types de mesure	38
4.1	Diagramme de cas d'utilisation du Sprint 2	40
4.2	Diagramme de séquence de la consultation du tableau de bord décisionnel	41
4.3	Tableau de bord IoT temps réel avec cartes d'indicateurs et séries temporelles	42
4.4	Cartographie Leaflet.js des capteurs avec marqueurs colorés selon le statut	42

5.1	Diagramme de cas d'utilisation du Sprint 3	46
5.2	Diagramme de séquence de la prédiction Prophet	47
5.3	Prévision Prophet à 7 jours avec intervalles de confiance	48
5.4	Diagramme de séquence de la détection d'anomalies (Isolation Forest)	48
6.1	Diagramme de cas d'utilisation du Sprint 4	51
6.2	Diagramme de séquence du déclenchement et de la notification d'une alerte . .	52
6.3	Formulaire de configuration d'une règle d'alerte	53
6.4	Gestion des alertes et notifications par e-mail	54
6.5	Exemple d'une notification d'alerte envoyée par courrier électronique	55
7.1	Diagramme de cas d'utilisation du Sprint 5	57
7.2	Courriel de création de compte envoyé automatiquement à l'utilisateur	58
7.3	Écran de changement obligatoire du mot de passe lors de la première connexion	58
7.4	Module d'administration des utilisateurs et de leurs rôles	59
7.5	Diagramme de séquence de l'assistant conversationnel (Llama 3.2)	60
7.6	Panneau de l'assistant conversationnel avec réponse en streaming	62
7.7	Diagramme de cas d'utilisation de la gestion des favoris	63
7.8	Diagramme de séquence de la gestion des favoris	63
7.9	Dépôt GitHub du projet Elastic Watch	65
7.10	Tableau de bord Vercel — déploiement du front-end React	66
7.11	Tableau de bord Railway — back-end Laravel, MySQL et Redis	66
7.12	Tableau de bord Render — microservices ML (Flask) et Chatbot (FastAPI) . .	67

Liste des tableaux

1.1	Tableau comparatif des solutions existantes	6
1.2	Équipe de travail selon les rôles Scrum	8
1.3	Product Backlog	9
1.4	Planification des sprints	12
1.5	Sprint Backlog — Sprint 1	13
2.1	Besoins non fonctionnels du système	17
2.2	Structure du Data Warehouse en schéma en étoile	26
2.3	Backlog produit — 12 User Stories priorisées	26
2.4	Tests unitaires des modules Authentification et Espaces (extrait)	34
3.1	Types de capteurs supportés par la plateforme	37
3.2	Tests unitaires des modules Passerelles et Capteurs (extrait)	38
4.1	Points d'entrée de l'API du tableau de bord IoT	41
4.2	Tests unitaires du module d'ingestion IoT (extrait)	43
5.1	Fiches descriptives des cas d'utilisation UC-11 et UC-12	44
5.2	Comparatif des modèles de prédiction (capteur de température)	49
5.3	Résultats de la détection d'anomalies (Isolation Forest)	49
6.1	Fiches descriptives des cas d'utilisation UC-13 et UC-14	51
6.2	Tests unitaires du système d'alertes (extrait)	55
7.1	Fiches descriptives des cas d'utilisation de la gestion des favoris	60
7.2	Services déployés via Docker Compose	64
7.3	Répartition des services sur les plateformes cloud (Vercel / Railway / Render)	65
7.4	Tests unitaires du Sprint 6 (extrait)	67
8.1	Synthèse de la couverture de tests	68
8.2	Indicateurs de performance de la détection d'anomalies	69
8.3	Tableau de conformité aux exigences non fonctionnelles	70
8.3	Tableau de conformité aux exigences non fonctionnelles	71
8.4	Synthèse des résultats clés mesurés	72
9.1	Glossaire des termes techniques	75

9.2	Checklist de déploiement avant soutenance	77
9.3	Liste des endpoints de l'API REST	78

Liste des abréviations

Sigle	Signification
API	Application Programming Interface
BI	Business Intelligence
DW	Data Warehouse
ETL	Extract, Transform, Load
IoT	Internet of Things (Internet des Objets)
JWT	JSON Web Token
KPI	Key Performance Indicator
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
MQTT	Message Queuing Telemetry Transport
MVC	Model-View-Controller
OTP	One-Time Password
PFE	Projet de Fin d'Études
REST	Representational State Transfer
RMSE	Root Mean Square Error
SPA	Single Page Application
SQL	Structured Query Language
TWIN	Technologies du Web et de l'Internet des Objets
UML	Unified Modeling Language

Introduction Générale

Depuis que les entreprises ont entamé leur transformation numérique, on assiste à une véritable explosion des données. Capteurs industriels, automates programmables, équipements réseau et applications métier produisent en continu d'énormes volumes d'informations. Les outils classiques ne suffisent plus pour tout traiter.

Dans le secteur des télécommunications particulièrement, il est devenu essentiel de suivre en temps réel la qualité de service, la consommation d'énergie et l'état des équipements répartis sur de vastes zones géographiques.

TAC-TIC est une société tunisienne basée au Pôle Technologique El Ghazela à Ariana. Spécialisée dans les solutions TIC et les infrastructures de télécommunication, elle fait face chaque jour à ce défi : ses clients déploient des infrastructures IoT complexes, mais les données sont dispersées dans plusieurs systèmes différents qui ne communiquent pas entre eux. Sans une plateforme centralisée capable d'analyser et de superviser l'ensemble, il est presque impossible de prendre des décisions de manière proactive.

C'est dans ce contexte que s'inscrit mon stage. Avec la transformation numérique et l'augmentation constante des données industrielles, la supervision et l'analyse en temps réel sont devenues un enjeu majeur pour les entreprises. L'objectif était donc de concevoir une plateforme simple, directement utilisable, basée sur des technologies bien maîtrisées et qui répond aux vrais besoins du terrain industriel.

La plateforme que j'ai développée permet de collecter des données provenant de différentes sources — capteurs, logs et bases de données hétérogènes — afin de les regrouper dans un seul environnement structuré. Cela permet de visualiser ces données presque en temps réel. J'ai également intégré de l'intelligence artificielle pour détecter à l'avance les anomalies et améliorer la prise de décision avant que les incidents ne se produisent.

Ce rapport est organisé en cinq chapitres principaux. Le Chapitre 1 présente l'état de l'art et l'analyse du marché, avec un aperçu des solutions existantes et le positionnement d'Elastic Watch. Le Chapitre 2 traite de l'analyse des besoins et de la conception du système (acteurs, besoins fonctionnels et non fonctionnels, modélisation UML et architecture technique). Le Chapitre 3 détaille le développement réalisé pendant les sprints. Le Chapitre 4 regroupe les tests, la validation et les résultats, notamment les tests unitaires (PHPUnit et Pytest), les tests d'intégration (Postman), les tests de performance (JMeter) et l'évaluation des modèles d'intelligence artificielle. Enfin, une conclusion générale revient sur les apports du projet et propose des perspectives d'évolution.

Chapitre 1

Contexte général du projet

1.1 Introduction

Dans ce premier chapitre, nous établissons les fondements du projet en offrant une vue d'ensemble ainsi qu'une introduction à l'entreprise d'accueil. Nous débutons par définir le cadre global du stage, situant ainsi notre travail dans son contexte approprié. Ensuite, nous présentons *TAC-TIC*, l'entreprise hôte, en détaillant ses services et domaines d'intervention. Après cette introduction, nous décrivons les grandes lignes du projet **Elastic Watch**, en énonçant ses objectifs principaux et en identifiant les défis à surmonter. Une étude préalable approfondie est menée, comprenant une analyse de l'existant, une critique des faiblesses relevées et la proposition de solutions adaptées. Pour conclure, nous détaillons la méthodologie adoptée, en justifiant le choix du cadre *Scrum* et en précisant les rôles qui y sont associés. Ce chapitre jette ainsi les bases nécessaires à la compréhension du projet.

1.2 Cadre général du stage

Dans le cadre de notre formation à **ESPRIT (École Supérieure Privée d'Ingénierie et de Technologies)**, nous sommes amenés à réaliser un projet de fin d'études afin de mettre en pratique les compétences acquises durant notre parcours académique. J'ai eu l'opportunité d'effectuer un stage de six mois chez **TAC-TIC**, en collaboration avec *Elastic Solutions*, où j'ai participé à la conception et au développement d'**Elastic Watch**, une plateforme de supervision et d'analyse des données en temps réel intégrant la centralisation des données et des mécanismes d'intelligence artificielle pour l'anticipation des anomalies.

1.3 Présentation de l'entreprise d'accueil — TAC-TIC

1.3.1 Présentation de TAC-TIC

TAC-TIC (Technology and Communications) est une entreprise tunisienne implantée au Pôle Technologique El Ghazela à Ariana. Elle est spécialisée dans l'intégration de solutions technologiques innovantes destinées à accompagner la transformation digitale des entreprises.

L'entreprise intervient principalement dans les domaines de l'Internet des Objets (IoT), de l'intelligence artificielle, de la supervision d'infrastructures, de la cybersécurité et de l'analyse des données. *TAC-TIC* accompagne ses clients dans la conception, le déploiement et l'exploitation de solutions intelligentes permettant d'améliorer le suivi des systèmes, la prise de décision et l'optimisation des performances. Dans le cadre de ses activités, *TAC-TIC* collabore avec *Elastic Solutions* afin de développer des plateformes avancées de monitoring et d'analyse de données en temps réel.



FIGURE 1.1 – Logo de TAC-TIC

1.3.2 Services proposés par TAC-TIC

TAC-TIC propose plusieurs services technologiques orientés vers la supervision et l'exploitation intelligente des données :

- **Intégration IoT** : déploiement de capteurs, équipements connectés et passerelles (*gateways*) pour la collecte des données.
- **Supervision intelligente** : suivi des infrastructures avec des mécanismes d'intelligence artificielle et de détection d'événements.
- **Analytics et Business Intelligence (BI)** : analyse des données et création de tableaux de bord pour le monitoring en temps réel.
- **Machine Learning et prédiction** : développement de modèles permettant d'anticiper les anomalies et d'améliorer la maintenance prédictive.
- **Transformation digitale et intégration de solutions TIC** : accompagnement des entreprises dans la modernisation de leurs systèmes d'information.

1.4 Présentation du projet Elastic Watch

1.4.1 Description générale

Elastic Watch est une plateforme de supervision et d'analyse des données en temps réel, conçue pour centraliser des données issues de sources hétérogènes et fournir une visualisation claire et interactive. Elle s'articule autour de quatre composantes majeures : un *Data Warehouse* structuré en schéma en étoile, des pipelines ETL automatisés, un module de visualisation interactive construit avec React.js et Chart.js, ainsi qu'un moteur d'intelligence artificielle intégrant des modèles Prophet pour la prédiction futures et la détection proactive des anomalies via Isolation Forest.

1.4.2 Problématique

Comment concevoir une plateforme de supervision et d'analyse centralisée capable de collecter des données hétérogènes (IoT, bases SQL, API, CSV), de les stocker de manière structurée et performante, de les visualiser en quasi-temps réel via des dashboards interactifs, et d'intégrer des modèles d'intelligence artificielle pour la détection et l'anticipation des anomalies ?

1.5 Étude de l'existant

Dans cette section, nous abordons une étude préliminaire qui se divise en trois volets principaux : l'analyse de l'existant, sa critique, et enfin la proposition de solutions.

1.5.1 Panorama des solutions existantes

Actuellement, il est souvent difficile de disposer d'une plateforme unifiée permettant de superviser et d'analyser des données issues de sources hétérogènes en temps réel. Pour répondre à ce besoin, plusieurs solutions de monitoring et de traitement de données ont été développées. Voici un panorama des systèmes existants dans le domaine de la supervision et de l'analyse de données.

Grafana

Grafana est devenu un standard de facto pour la visualisation de métriques et de séries temporelles. Sa communauté active, son extensibilité via des plugins et la richesse de ses options de dashboards en font un outil redoutable pour la présentation des données. Cependant, *Grafana* se limite à la couche de visualisation : il n'intègre ni ETL, ni *data warehouse* structuré, ni module d'intelligence artificielle embarqué. Pour disposer d'une chaîne complète, il faut assembler manuellement *Telegraf*, *InfluxDB* et *Kapacitor*, créant un empilement technologique difficile à maintenir.

AWS IoT Analytics

AWS IoT Analytics propose un service cloud clé en main pour traiter des données IoT dans l'écosystème Amazon. Il embarque l'ensemble du pipeline, de la récupération des données jusqu'aux tableaux de bord, et se connecte sans friction aux autres outils AWS comme *SageMaker* pour le ML, *QuickSight* pour la visualisation, ou *S3* pour le stockage. Cependant, cette facilité a un coût : dépendance totale à l'écosystème Amazon rendant toute migration coûteuse, expertise AWS difficile à trouver sur le marché tunisien, et facturation à l'usage pouvant devenir prohibitive à grande échelle.

Kibana / ELK Stack

La pile *ELK* — *Elasticsearch*, *Logstash*, *Kibana* — est très présente dans les projets d'analyse de logs. *Kibana* permet de représenter graphiquement de nombreuses données, et le langage *KQL* offre une exploration fine. *Logstash* ingère la plupart des formats et les transforme, ce qui est pratique face à des sources hétérogènes. Cependant, cette pile est conçue à l'origine pour la recherche de logs et de texte, non pour superviser des capteurs IoT en temps réel. Les requêtes d'agrégation sur des volumes structurés restent moins performantes qu'une base SQL optimisée, et les capacités de *Machine Learning* prédictif nécessitent des briques externes.

Home Assistant

Home Assistant est une solution open-source largement utilisée dans le domaine de la domotique pour la gestion et l'automatisation des équipements connectés à domicile. Elle permet de centraliser des capteurs, de créer des automatisations simples et de visualiser l'état des dispositifs via une interface conviviale. Cependant, malgré sa popularité, cette solution n'est pas adaptée aux environnements industriels : elle montre ses limites en termes de scalabilité, de gestion de gros volumes de données, de haute disponibilité et de traitement en temps réel à grande échelle. Elle n'intègre pas nativement de mécanismes avancés d'analyse ou de *Machine Learning* pour la détection d'anomalies dans des systèmes industriels complexes.

1.5.2 Tableau comparatif global

Le Tableau 1.1 synthétise les forces et limites des solutions étudiées face aux critères retenus pour le projet *Elastic Watch*.

TABLE 1.1 – Tableau comparatif des solutions existantes

Critère	Grafana	AWS IoT	Kibana	Elastic Watch
ML intégré	*	Partiel	*	✓ Prophet+IF
Data Warehouse	*	✓	*	✓ Star Schema
ETL automatisé	*	✓	Partiel	✓ Pandas+Scheduler
Alertes métier	Partiel	✓	Partiel	✓ Multi-canal
RGPD / On-premise	✓	*	✓	✓ Local + cloud
Coût	Gratuit/Pro	Élevé	Gratuit/Pro	Open-source

Conclusion : aucune des solutions étudiées n'intègre simultanément la gestion spatiale hiérarchique, le monitoring IoT+IA unifié, les règles d'alerting métier configurables et le module ML prédictif multi-modèles dans une interface web accessible et déployable en entreprise. C'est précisément ce vide qu'*Elastic Watch* vient combler.

1.5.3 Solution proposée — Elastic Watch

À la suite de notre analyse approfondie du contexte industriel et des besoins en supervision de données en temps réel, nous proposons la conception et le développement d'une plateforme de supervision intelligente nommée **Elastic Watch**. Cette solution vise à centraliser les données provenant de sources hétérogènes (capteurs IoT, bases de données, API et logs) au sein d'un système unifié, afin de garantir une exploitation fluide, cohérente et en temps réel.

Notre mission inclut :

- Un *Data Warehouse* structuré en schéma en étoile pour l'analytique multidimensionnelle haute performance.
- Des pipelines ETL automatisés (Pandas + Laravel Scheduler) pour la transformation et le chargement des données.
- Un tableau de bord BI interactif (React.js + Chart.js + Leaflet) pour la visualisation en temps réel.
- Un moteur d'IA combinant Prophet et Isolation Forest pour la prédiction et la détection d'anomalies.
- Un système d'alertes configurables avec notifications multi-canaux (email + push).
- Une architecture scalable et performante adaptée aux environnements industriels.

1.6 Méthodologie de travail

Dans cette section, nous analysons les approches agiles pour garantir l'atteinte des objectifs fixés. Contrairement aux méthodes traditionnelles de conception logicielle, les méthodologies agiles se montrent plus pragmatiques et encouragent une participation active du client tout au long du processus de développement. De plus, elles ont prouvé leur capacité à augmenter la productivité et à réduire les délais de livraison.

1.6.1 Choix du methode Scrum

Pour mener à bien ce projet, nous avons adopté la méthodologie agile *Scrum*, particulièrement adaptée aux projets intégrant de multiples technologies et nécessitant des livraisons régulières. *Scrum* est structuré autour de cycles d'itérations répétitifs appelés *Sprints*. Les raisons ayant conduit à ce choix sont les suivantes :

- **Cadre structuré** : *Scrum* offre une structure claire avec des rôles définis et des événements réguliers, facilitant la planification et l'exécution des projets.
- **Flexibilité et adaptation** : il permet de s'adapter rapidement aux changements grâce à des itérations courtes et à la priorisation des fonctionnalités en fonction des besoins.
- **Amélioration continue** : des rétrospectives fréquentes permettent à l'équipe de tirer des enseignements de chaque itération.
- **Communication et transparence** : *Scrum* encourage une communication ouverte entre les membres de l'équipe et les parties prenantes, facilitant l'alignement des attentes et la résolution rapide des problèmes.



FIGURE 1.2 – La méthode Scrum

1.6.2 Les rôles dans la méthodologie Agile

La méthodologie *Scrum* s'appuie sur la collaboration de trois acteurs clés, présentés dans le Tableau 1.2.

TABLE 1.2 – Équipe de travail selon les rôles Scrum

Rôle	Responsable / Description
Scrum Master	Supervise l'avancement du projet, organise les réunions Scrum, facilite le processus et élimine les obstacles.
Product Owner	Chargé de maximiser la valeur du projet ; gère le Backlog et priorise les exigences selon les besoins des parties prenantes.
Scrum Team (Développeur)	Elaa SBOUI — conception, développement et tests de la plateforme Elastic Watch.

Chez **TAC-TIC**, mon encadrant entreprise est **M. Mohamed Hedi GHARBI**. Il assure le suivi du projet *Elastic Watch* et veille à la bonne progression des différentes phases de développement, en garantissant l'adéquation de la solution avec les besoins industriels et les objectifs du stage.

1.6.3 Product Backlog

Le *Product Backlog* regroupe l'ensemble des *User Stories* du projet, définies et priorisées par le *Product Owner*. Il constitue la base de la planification de chaque sprint. Le Tableau 1.3 présente le *Product Backlog* du projet *Elastic Watch*.

TABLE 1.3 – Product Backlog

ID	Thème	ID US	Prio.	User Story	SP	Release	Sprint	Statut
1	Authentification JWT + OTP	1.1	1	En tant qu'utilisateur je veux m'authentifier avec mes identifiants pour obtenir un token JWT	6	1	1	Terminé
		1.2	1	En tant qu'utilisateur je veux réinitialiser mon mot de passe via un code OTP à 6 chiffres envoyé par email (expiration 10 min)	5	1	1	Terminé
		1.3	2	En tant qu'utilisateur je veux être forcé à changer mon mot de passe à la première connexion	3	1	1	Terminé
2	Gestion des utilisateurs & RBAC	2.1	1	En tant qu'administrateur je veux créer un utilisateur et lui assigner un rôle	5	1	1	Terminé
		2.2	2	En tant qu'administrateur je veux activer, désactiver ou supprimer un compte utilisateur	4	1	1	Terminé
		2.3	2	En tant qu'administrateur je veux gérer les rôles et les permissions (RBAC)	5	1	1	Terminé
3	Gestion des profils	3.1	3	En tant qu'utilisateur je veux consulter mon profil (photo, nom, rôle)	3	1	1	Terminé
		3.2	3	En tant qu'utilisateur je veux modifier mon profil et ma photo de profil	4	1	1	Terminé
4	Gestion des espaces IoT	4.1	1	En tant qu'administrateur je veux créer et modifier un espace (bâtiment, salle, zone) avec hiérarchie parent/enfant	6	1	1	Terminé
		4.2	2	En tant qu'utilisateur je veux consulter l'arborescence des espaces et leurs statistiques (devices, alertes)	4	1	1	Terminé

Product Backlog (suite)

ID	Thème	ID US	Prio.	User Story	SP	Release	Sprint	Statut
		4.3	2	En tant qu'administrateur je veux associer un plan d'espace et dessiner un polygone géographique	5	1	1	Terminé
5	Gestion des Gateways IoT	5.1	1	En tant qu'administrateur je veux enregistrer une Gateway IoT/AI et générer son token API (64 caractères)	5	1	2	Terminé
		5.2	2	En tant qu'administrateur je veux modifier et activer/désactiver une Gateway	3	1	2	Terminé
		5.3	3	En tant qu'utilisateur je veux consulter la liste des Gateways et leurs devices associés	3	1	2	Terminé
6	Gestion des Devices	6.1	1	En tant qu'administrateur je veux enregistrer un device (capteur, caméra) et choisir son type	5	1	2	Terminé
		6.2	1	En tant qu'administrateur je veux modifier, activer ou désactiver un device	3	1	2	Terminé
		6.3	2	En tant qu'administrateur je veux associer un device à une position sur le plan d'espace	4	1	2	Terminé
7	Surveillance & Ingestion de données	7.1	1	En tant que gateway IoT je veux envoyer des mesures capteurs via l'API d'ingestion	6	2	3	Terminé
		7.2	1	En tant qu'utilisateur je veux visualiser les lectures des capteurs en temps réel sur le dashboard	5	2	3	Terminé
		7.3	2	En tant qu'utilisateur je veux consulter l'historique des mesures par device et par période	4	2	3	Terminé
8	Règles & Alertes	8.1	1	En tant qu'administrateur je veux créer une règle d'alerte basée sur un indice et une condition seuil	6	2	3	Terminé
		8.2	1	En tant qu'utilisateur je veux recevoir une notification temps réel lorsqu'une règle est violée	5	2	3	Terminé
		8.3	2	En tant qu'utilisateur je veux consulter et marquer comme lues mes notifications d'alerte	3	2	3	Terminé

Product Backlog (suite)

ID	Thème	ID US	Prio.	User Story	SP	Release	Sprint	Statut
9	Gestion des indices	9.1	2	En tant qu'administrateur je veux créer un indice de surveillance combinant plusieurs capteurs	5	2	3	Terminé
		9.2	3	En tant qu'administrateur je veux configurer les capteurs et la formule de calcul d'un indice	4	2	3	Terminé
10	Entrepôt de données BI	10.1	2	En tant que système je veux que les mesures capteurs soient automatiquement chargées dans le schéma étoile via ETL (Laravel Scheduler)	6	2	4	Terminé
		10.2	3	En tant qu'utilisateur je veux consulter des tableaux de bord analytiques agrégés (par espace, device, période)	5	2	4	Terminé
11	Prédiction (Prophet)	11.1	2	En tant qu'utilisateur je veux voir une prédiction des valeurs capteurs sur 7 jours (modèle Prophet, MAPE 2,3,%)	7	2	4	Terminé
		11.2	3	En tant qu'utilisateur je veux visualiser la courbe de prédiction et l'intervalle de confiance	4	2	4	Terminé
12	Détection d'anomalies (Isolation Forest)	12.1	1	En tant qu'utilisateur je veux que les anomalies soient détectées automatiquement par Isolation Forest (F1 91,8 %)	7	2	4	Terminé
		12.2	2	En tant qu'utilisateur je veux visualiser les anomalies détectées sur le graphique des mesures	4	2	4	Terminé
13	Assistant conversationnel	13.1	3	En tant qu'utilisateur je veux interroger un chatbot IA pour obtenir des informations sur l'état de mon infrastructure IoT	7	3	5	Terminé
		13.2	4	En tant qu'utilisateur je veux que le chatbot dispose du contexte temps réel (espaces, devices, alertes) pour répondre avec précision	5	3	5	Terminé
14	Génération de polygones (PolygonNet)	14.1	3	En tant qu'administrateur je veux qu'un polygone de position soit généré automatiquement par le réseau de neurones PolygonNet	7	3	5	Terminé

Product Backlog (suite)

ID	Thème	ID US	Prio.	User Story	SP	Release	Sprint	Statut
		14.2	4	En tant qu'administrateur je veux ajuster manuellement le polygone généré par Polygon-Net	3	3	5	Terminé
15	Déploiement & Conteneurisation	15.1	2	En tant que DevOps je veux conteneuriser tous les services (Laravel, React, Flask, FastAPI) avec Docker Compose	6	3	6	Terminé
		15.2	3	En tant que DevOps je veux déployer la plateforme sur un environnement de production	5	3	6	Terminé

1.6.4 Planification des sprints

Le projet est découpé en sprints de deux semaines. Le Tableau 1.4 présente la planification et la répartition des fonctionnalités sur les différents sprints.

TABLE 1.4 – Planification des sprints

Sprint	Objectif principal	Durée	Charge
Sprint 1	Authentification JWT+OTP + Gestion des espaces	2 semaines	22 h
Sprint 2	Passerelles IoT/AI + Dispositifs capteurs	2 semaines	26 h
Sprint 3	Ingestion IoT + Dashboard BI + ETL	2 semaines	36 h
Sprint 4	Intelligence Artificielle (Prophet, Isolation Forest)	2 semaines	34 h
Sprint 5	Système d'alertes & Supervision multi-canal	2 semaines	26 h
Sprint 6	Admin + Chatbot Llama 3.2 + Favoris + Docker + Cloud	2 semaines	32 h

1.6.5 Sprint Backlog

Le *Sprint Backlog* détaille, pour chaque sprint, les tâches concrètes à réaliser, leur estimation en points de story (*SP*) et le responsable assigné.

TABLE 1.5 – Sprint Backlog — Sprint 1

Thème	ID	User Story	ID tâche	Tâche	SP	Responsable
Authentification JWT + OTP	1.1	En tant qu'utilisateur, je veux m'authentifier avec mes identifiants afin d'obtenir un token JWT et accéder à la plateforme	1.1.1	Réalisation des diagrammes de cas d'utilisation, de classe et de séquence pour « s'authentifier »	2	Sboui Elaa
			1.1.2	Développement du login (vérification email/mot de passe, génération token JWT via Laravel Passport, gestion is_active)	3	Sboui Elaa
			1.1.3	Tests unitaires et d'intégration du cas « s'authentifier » (PHPUnit)	1	Sboui Elaa
	1.2	En tant qu'utilisateur, je veux réinitialiser mon mot de passe via un code OTP à 6 chiffres envoyé par email	1.2.1	Réalisation des diagrammes de cas d'utilisation, de classe et de séquence pour « réinitialiser mot de passe via OTP »	1	Sboui Elaa
			1.2.2	Développement OTP : génération hash, envoi email, vérification 6 chiffres, expiration 10 min, consommation du code (OtpVerification)	3	Sboui Elaa
			1.2.3	Tests du flux OTP (code correct, expiré, déjà consommé)	1	Sboui Elaa
	1.3	En tant qu'utilisateur, je veux être forcé à changer mon mot de passe lors de ma première connexion	1.3.1	Réalisation des diagrammes pour « changement de mot de passe forcé »	1	Sboui Elaa
			1.3.2	Développement du middleware force_password_change et de l'endpoint de mise à jour	1	Sboui Elaa
			1.3.3	Tests du cas « changement de mot de passe forcé »	1	Sboui Elaa
Gestion des espaces	2.1	En tant qu'administrateur, je veux créer et modifier un espace (bâtiment, salle, zone) avec sa hiérarchie parent/enfant	2.1.1	Réalisation des diagrammes de cas d'utilisation, de classe et de séquence pour « créer / modifier un espace »	2	Sboui Elaa

Sprint Backlog — Sprint 1 (suite)

Thème	ID	User Story	ID tâche	Tâche	SP	Responsable
			2.1.2	Développement CRUD espace : création, modification, suppression, hiérarchie parent_id, type, pays, statut	3	Sboui Elaa
			2.1.3	Tests CRUD espaces (SpaceControllerTest — PHPU-nit)	1	Sboui Elaa
	2.2	En tant qu'utilisateur, je veux consulter la liste des espaces et naviguer dans leur arborescence	2.2.1	Réalisation des diagrammes pour « consulter les espaces »	1	Sboui Elaa
			2.2.2	Développement de l'affichage arborescence des espaces (sous-espaces récur-sifs), compteur devices, statut alerte	2	Sboui Elaa
			2.2.3	Tests de consultation et filtrage des espaces	1	Sboui Elaa
	2.3	En tant qu'administrateur, je veux associer un plan d'espace (image ou polygone dessiné) à un espace	2.3.1	Réalisation des diagrammes pour « gérer le plan d'espace »	1	Sboui Elaa
			2.3.2	Développement upload plan image (plan_image), dessin et sauvegarde polygone JSON (drawn_polygon, selected_place_polygon)	3	Sboui Elaa
			2.3.3	Tests du plan d'espace (upload, polygone, coordonnées GPS)	1	Sboui Elaa

1.7 Conclusion

Dans ce premier chapitre, nous avons présenté une vue d'ensemble du projet **Elastic Watch**. Nous avons d'abord défini le cadre général du stage ainsi que l'organisme d'accueil **TAC-TIC**, en mettant en avant son domaine d'activité et ses services. Ensuite, nous avons introduit le projet en précisant ses objectifs principaux ainsi que la problématique à laquelle il répond.

Nous avons également réalisé une étude préliminaire comprenant une analyse des solutions existantes (*ThingsBoard*, *Grafana*, *AWS IoT Analytics*, *Kibana* et *Home Assistant*), la critique de leurs limites, ainsi que la solution proposée pour améliorer la supervision et l'analyse des données en temps réel. Cette présentation permet de poser les bases nécessaires pour la suite du travail et d'aborder les prochaines phases du projet de manière structurée et cohérente.

Chapitre 2

Sprint 0 — Analyse et Spécification des Besoins

2.1 Introduction

Ce chapitre constitue le socle méthodologique et conceptuel du projet. Il présente successivement les besoins fonctionnels et non fonctionnels, la modélisation UML du système, l'environnement de travail, l'architecture retenue et le *backlog produit*. Il détaille enfin la réalisation du premier sprint, consacré à l'authentification sécurisée et à la gestion hiérarchique des espaces.

2.2 Identification des acteurs

Le système *Elastic Watch* met en jeu quatre acteurs, dont trois profils humains et un acteur système :

- **Administrateur** : dispose de l'ensemble des droits. Il configure les espaces, les passerelles, les capteurs, les règles d'alerte et gère les comptes utilisateurs.
- **Technicien** : exploite les tableaux de bord, consulte les prédictions et les anomalies, et acquitte les alertes.
- **Consultant (Viewer)** : accède en lecture seule aux tableaux de bord et aux indicateurs.
- **Passerelle IoT (acteur système)** : n'accède qu'au point d'entrée d'ingestion des mesures.

L'Administrateur hérite de l'intégralité des droits du Technicien par généralisation.

2.3 Spécification des besoins

2.3.1 Besoins fonctionnels

La plateforme doit permettre l'authentification à deux facteurs (JWT + OTP), la gestion hiérarchique des espaces, l'enregistrement et la configuration des passerelles IoT et AI, la gestion des capteurs et de leurs types de mesure, l'ingestion des mesures en mode unitaire et en lot, la

visualisation temps réel via des tableaux de bord interactifs, la prédiction des tendances au moyen de modèles d'apprentissage automatique, la détection d'anomalies, la configuration de règles d'alerte dynamiques, la notification multi-canal et l'administration des comptes utilisateurs.

2.3.2 Besoins non fonctionnels

Les exigences non fonctionnelles, mesurables et vérifiables, sont récapitulées dans le Tableau 2.1.

TABLE 2.1 – Besoins non fonctionnels du système

Critère	Exigence mesurable
Performance	Temps de réponse du tableau de bord inférieur à 2 000 ms pour 95 % des requêtes (50 utilisateurs concurrents). Cache Redis avec TTL de 5 minutes.
Scalabilité	Ingestion IoT d'au moins 1 000 messages par seconde. Architecture horizontalement scalable via Docker.
Sécurité	TLS sur l'ensemble des communications. JWT HS256 avec expiration à 15 minutes et rafraîchissement à 7 jours. Mots de passe hachés en Bcrypt. OTP par courriel (5 minutes, stocké dans Redis). Journal d'audit complet.
Maintenabilité	Couverture de tests supérieure à 70 %. Déploiement Docker reproductible. Documentation d'API sous Postman. Code versionné sur GitHub.
Ergonomie	Interface responsive (poste de travail et tablette). Prise en main inférieure à 30 minutes pour un analyste métier.

2.4 Modélisation UML

2.4.1 Diagramme des cas d'utilisation global

Le diagramme global présente les interactions entre les quatre acteurs et les cas d'utilisation organisés en paquets : Authentification, Supervision IoT, Analytique ML, Administration et Alerting. La passerelle IoT est un acteur système dont l'accès se limite au point d'entrée d'ingestion.

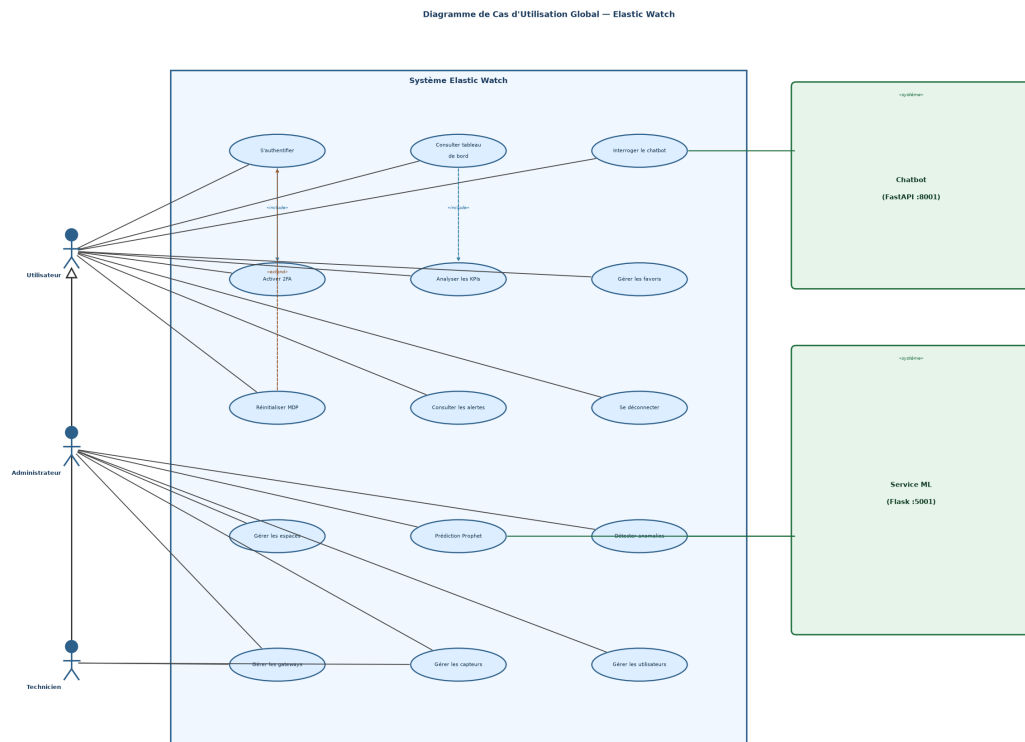


FIGURE 2.1 – Diagramme des cas d'utilisation global du système Elastic Watch

2.4.2 Diagramme de classes

La couche métier d'*Elastic Watch* s'articule autour de neuf entités principales, dont le rôle et les attributs clés sont détaillés dans l'image .

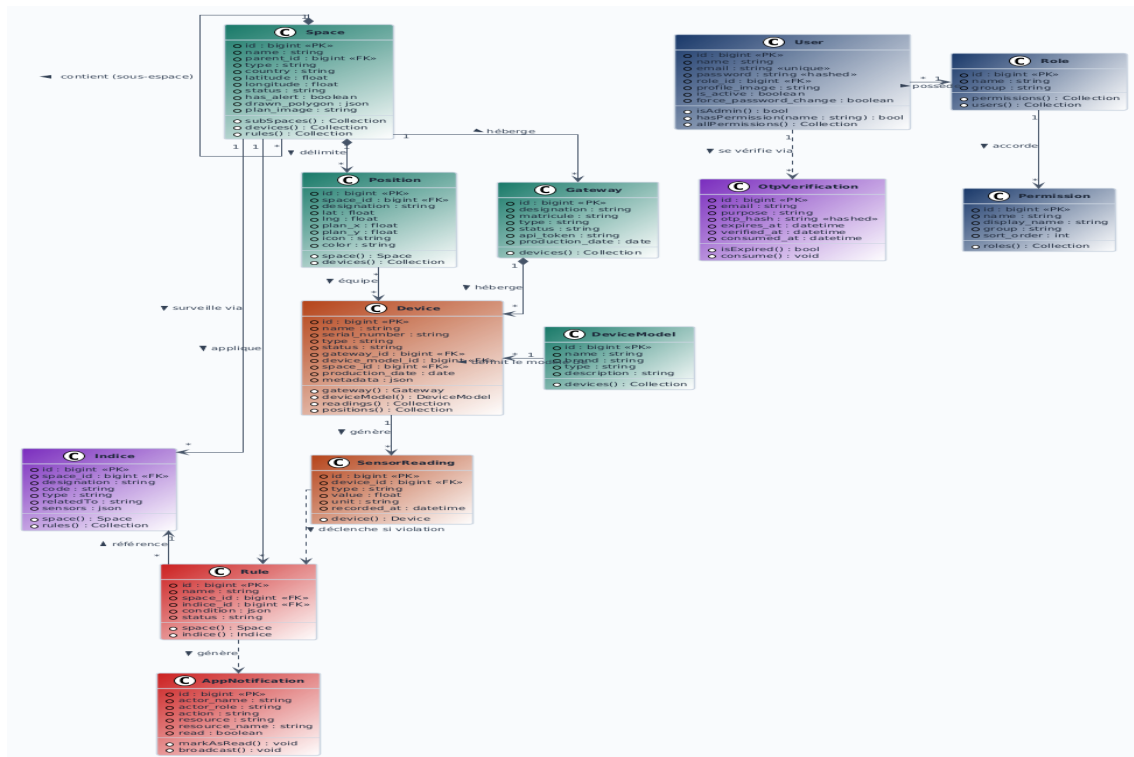


FIGURE 2.2 – Diagramme de classes de la couche métier d'Elastic Watch

2.5 Environnement de travail

2.5.1 Frameworks et langages utilisés

- **React.js** : bibliothèque JavaScript développée par Meta pour créer des interfaces utilisateur dynamiques et réactives.
- **Tailwind CSS** : framework CSS utilitaire permettant de construire rapidement des interfaces à partir de classes prédéfinies, sans écrire de CSS personnalisé.
- **Laravel 12** : framework PHP utilisé pour développer le back-end et structurer la logique métier de manière sécurisée et scalable.
- **Laravel Passport** : système d'authentification fondé sur OAuth2, permettant de sécuriser les API et de gérer les jetons d'accès.
- **Chart.js** : bibliothèque JavaScript utilisée pour créer des graphiques interactifs dans les tableaux de bord.
- **Python et Pandas** : langage et bibliothèque utilisés pour le traitement, la manipulation et l'analyse des données.

2.5.2 Environnement logiciel

- **Render** : plateforme cloud utilisée pour le déploiement des applications et des services backend. Elle permet un hébergement fiable, un déploiement automatique à partir d'un dépôt Git et une intégration continue (CI/CD).



FIGURE 2.3 – Logo Render

- **Railway** : plateforme de déploiement cloud permettant d'héberger des applications, des services backend et des bases de données grâce à une configuration simple et un déploiement automatisé.



FIGURE 2.4 – Logo Railway

- **Vercel** : plateforme de déploiement spécialisée dans les applications web modernes. Elle est utilisée pour héberger les applications React et offre un déploiement continu, une distribution via CDN et des performances optimisées.



FIGURE 2.5 – Logo Vercel

- **MySQL** : système de gestion de base de données relationnelle utilisé pour stocker et organiser les données de manière structurée.



FIGURE 2.6 – Logo MySQL

- **Postman** : outil de développement et de test d'API offrant une interface graphique pour créer, tester et gérer des requêtes HTTP.



FIGURE 2.7 – Logo Postman

- **Docker** : plateforme de conteneurisation permettant de développer, expédier et exécuter les applications de manière cohérente et portable, en encapsulant l'application et ses dépendances dans un environnement isolé.



FIGURE 2.8 – Logo Docker

- **GitHub** : plateforme de développement collaboratif fondée sur Git, permettant de stocker, gérer et collaborer sur le code source.



FIGURE 2.9 – Logo GitHub

- **PlantUML** : outil de modélisation UML utilisé outil de modélisation UML permettant de créer rapidement des diagrammes UML à partir de descriptions textuelles, facilitant ainsi la conception, la documentation et la représentation de l'architecture du système.

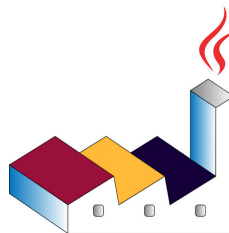


FIGURE 2.10 – Logo PlantUML

- **Visual Studio Code** : éditeur de code source léger et extensible utilisé pour le développement du projet.



FIGURE 2.11 – Logo Visual Studio Code

- **XAMPP** : environnement de développement local permettant d'exécuter Apache, MySQL et PHP pour tester le back-end.



FIGURE 2.12 – Logo XAMPP

2.6 Architecture du système

2.6.1 Architecture logique

L'architecture logique organise la solution en couches distinctes : présentation, logique métier, services d'apprentissage automatique et persistance. Cette séparation des préoccupations facilite la maintenance et autorise une montée en charge indépendante de chaque couche.

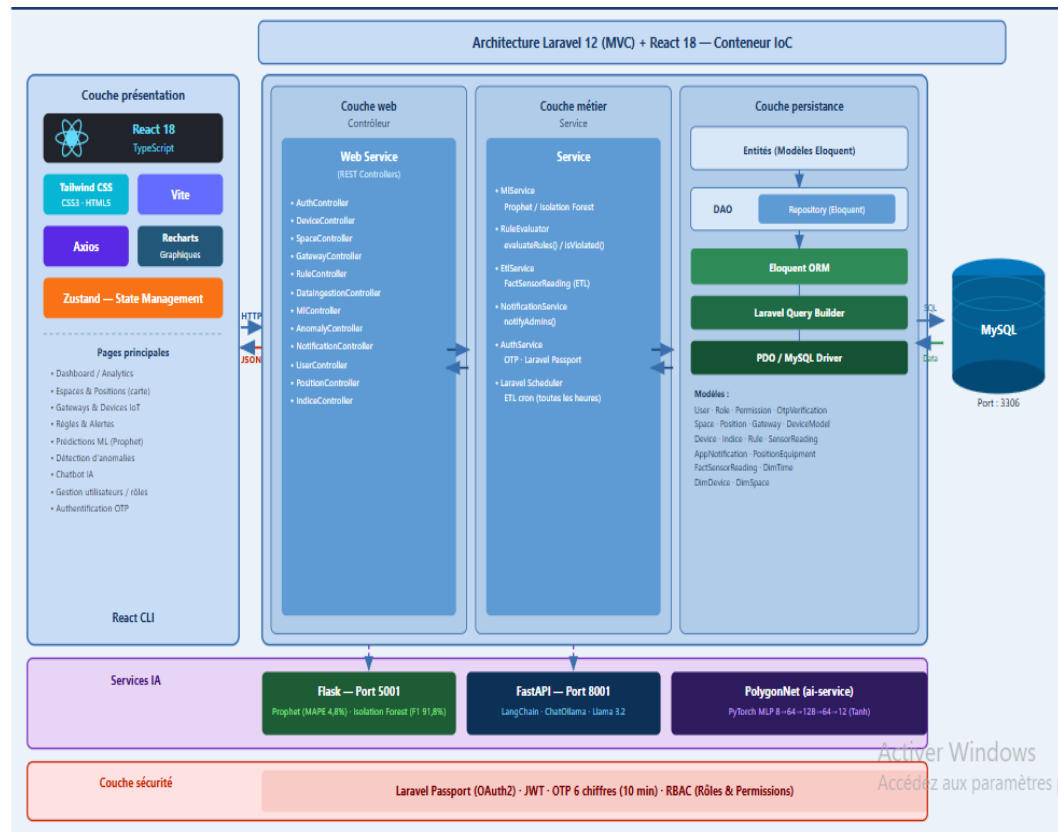


FIGURE 2.13 – Architecture logique de la plateforme Elastic Watch

2.6.2 Architecture physique

Elastic Watch repose sur cinq composants fonctionnant simultanément — trois serveurs applicatifs et deux systèmes de stockage :

- **React (Frontend) — port 5173** : application monopage (SPA) assurant l'interface utilisateur, les graphiques, les tableaux et la cartographie Leaflet.
- **Laravel 12 (Backend) — port 8000** : cœur du système, exposant l'API REST, évaluant les règles d'alerte, orchestrant l'ETL et les appels aux modèles ML.
- **Flask ML (Python) — port 5001** : microservice d'intelligence artificielle hébergeant Prophet et Isolation Forest.
- **MySQL 8.0 — port 3306** : stockage principal et *Data Warehouse* en schéma en étoile.

- **Redis 7 — port 6379** : cache (TTL 5 minutes), stockage des codes OTP et des sessions temporaires.

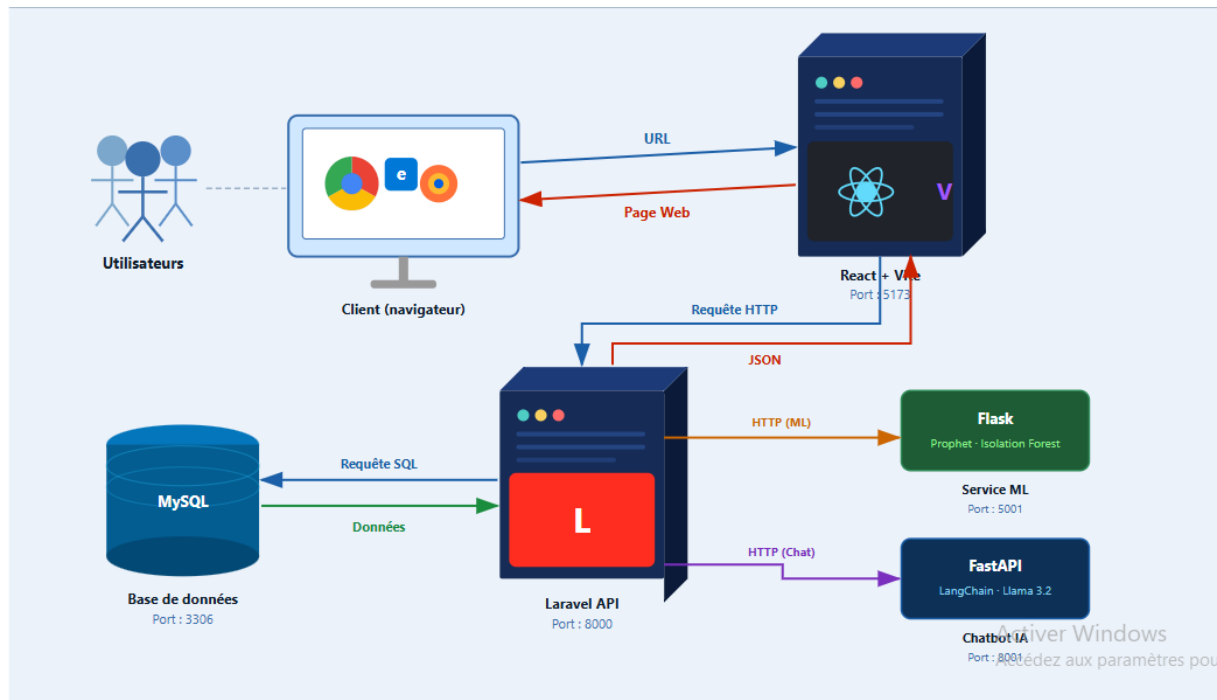


FIGURE 2.14 – Architecture physique de déploiement d'Elastic Watch

2.6.3 Data Warehouse — schéma en étoile

Le *Data Warehouse* adopte un schéma en étoile composé d'une table de faits centrale et de quatre tables de dimensions. Ce modèle garantit des performances OLAP optimales : les jointures directes autorisent des agrégations multidimensionnelles rapides, même sur des dizaines de millions de mesures. L'agrégation horaire réduit 10 000 lectures brutes à une seule ligne par heure et par capteur, ce qui accélère considérablement les requêtes analytiques.

TABLE 2.2 – Structure du Data Warehouse en schéma en étoile

Table	Type	Colonnes principales
fact_sensor_readings	FAITS	<i>reading_id</i> (PK), <i>avg_value</i> , <i>min_value</i> , <i>max_value</i> , <i>count</i> , <i>is_anomaly</i> , <i>device_id</i> (FK), <i>time_id</i> (FK), <i>space_id</i> (FK), <i>source_id</i> (FK).
dim_time	DIMENSION	<i>timestamp</i> , heure, jour, mois, année, semaine, jour de la semaine, indicateur de jour férié.
dim_device	DIMENSION	Nom du capteur, type, modèle, fabricant, <i>firmware</i> , passerelle associée.
dim_space	DIMENSION	Zone, type (site / bâtiment / salle), zone parente, latitude, longitude.
dim_source	DIMENSION	Type (MQTT / HTTP / API), configuration JSON, protocole.

2.7 Pilotage du projet avec Scrum

2.7.1 Backlog produit

Le *backlog produit*, défini et priorisé par le *Product Owner*, regroupe les douze *user stories* du projet. Il constitue la base de la planification des sprints, comme le montre la Figure ?? et le Tableau 2.3.

TABLE 2.3 – Backlog produit — 12 User Stories priorisées

ID	User Story	Priorité	Sprint
US01	Admin : s’authentifier via JWT + OTP courriel (deux facteurs).	Critique	S1
US02	Admin : gérer la hiérarchie des espaces (CRUD + carte Leaflet).	Élevée	S1
US03	Admin : enregistrer et configurer les passerelles IoT / IA.	Élevée	S2
US04	Admin : gérer les capteurs et leurs types de mesure.	Élevée	S2
US05	Technicien : visualiser le tableau de bord IoT temps réel avec KPI.	Critique	S3
US06	Passerelle : ingérer des mesures capteurs via API (unitaire et lot).	Critique	S3
US07	Technicien : consulter le tableau de bord IA (événements caméras).	Élevée	S4
US08	Technicien : visualiser le prédiction Prophet vs. réel.	Critique	S4

ID	User Story	Priorité	Sprint
US09	Système : détecter les anomalies et déclencher les alertes.	Critique	S5
US10	Admin : créer et gérer les règles d’alerte dynamiques.	Élevée	S5
US11	Admin : gérer les utilisateurs et leurs rôles.	Moyenne	S6
US12	Système : ETL automatisé vers le Data Warehouse (ordonnanceur horaire).	Critique	S6

2.7.2 Sprint Backlog

Le *Sprint Backlog* détaille, pour chaque sprint, les tâches concrètes à réaliser, leur estimation en points de story et le responsable assigné, comme l’illustre la Figure ??.

2.8 Sprint 0 — Authentification et gestion des espaces

2.8.1 Analyse des besoins

Le premier sprint couvre cinq cas d’utilisation principaux : la connexion au système, la vérification du code à usage unique, la réinitialisation du mot de passe, l’attribution des rôles et la gestion des espaces. La Figure 2.15 présente le diagramme de cas d’utilisation associé, dans lequel apparaissent les trois profils d’utilisateurs — administrateur, technicien et consultant — ainsi que le système lui-même, sollicité lors de la vérification du code à usage unique.

Le *Sprint Backlog* correspondant regroupe les récits utilisateurs suivants : « en tant qu’utilisateur, je veux me connecter avec mon courriel et mon mot de passe afin d’accéder à la plateforme » ; « en tant qu’utilisateur, je veux recevoir un code à usage unique par courriel pour valider ma connexion » ; « en tant qu’utilisateur, je veux pouvoir réinitialiser mon mot de passe en cas d’oubli » ; « en tant qu’administrateur, je veux créer des espaces organisés hiérarchiquement (bâtiments, étages, salles) » ; et « en tant qu’utilisateur, je veux me déconnecter de façon sécurisée ».

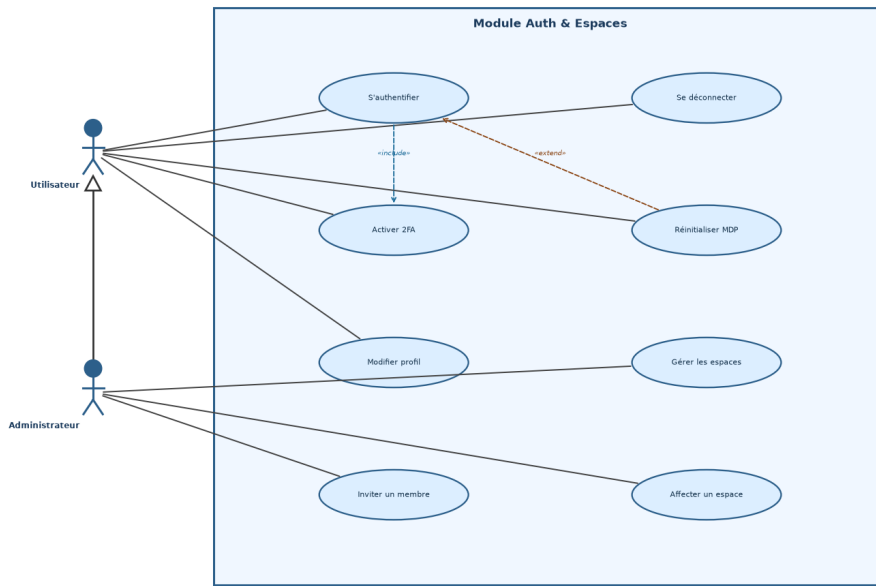


FIGURE 2.15 – Diagramme de cas d'utilisation du Sprint 1

2.8.2 Conception

La conception du mécanisme d'authentification a fait l'objet d'une attention particulière, la sécurité constituant une exigence non fonctionnelle prioritaire. Nous avons opté pour une authentification en deux phases combinant la vérification classique des identifiants et une seconde vérification par code à usage unique (*One-Time Password*). La Figure 2.16 illustre le déroulement complet de ce flux au moyen d'un diagramme de séquence.

Le processus d'authentification a été conçu en deux phases distinctes pour maximiser la sécurité. Tout commence par la saisie classique de l'e-mail et du mot de passe. Côté serveur, la vérification s'appuie sur l'algorithme Bcrypt, qui compare le mot de passe reçu avec l'empreinte hachée en base de données. Si cette première barrière est franchie, le système génère un code de vérification à six chiffres. Ce jeton temporaire est alors stocké dans Redis pour une durée de 300 secondes, le temps d'être transmis à l'utilisateur par mail.

La seconde étape intervient lorsque l'utilisateur soumet ce code. Le système effectue alors une comparaison directe avec la valeur en cache dans Redis. Une fois la correspondance validée, le code est immédiatement purgé du cache et un jeton JWT est créé via Laravel Passport pour sécuriser la session. Pour parer aux tentatives d'intrusion, j'ai intégré une gestion d'erreurs rigoureuse : chaque échec (code expiré, mauvais identifiants, etc.) renvoie une réponse HTTP spécifique, type 401 ou 422.

Enfin, concernant la réinitialisation des mots de passe (voir Figure 2.7), j'ai opté pour un jeton unique de 64 caractères, valide durant une heure. Ce choix permet d'éviter tout échange de données sensibles en clair. Pour protéger l'anonymat des utilisateurs et empêcher l'énumération

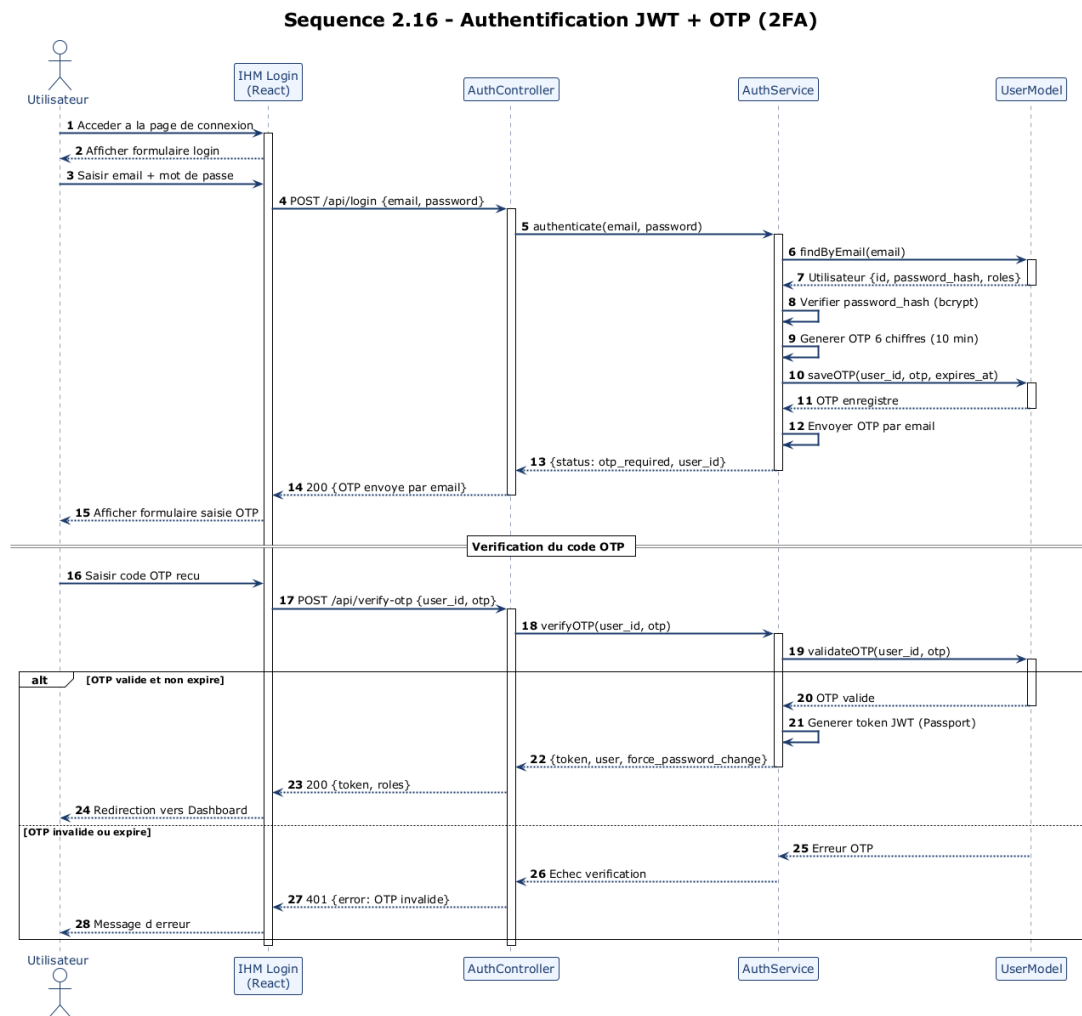


FIGURE 2.16 – Diagramme de séquence de l’authentification à deux facteurs (JWT + OTP)

de comptes par des tiers, le système affiche toujours un message générique, que l'adresse e-mail soit enregistrée ou non.

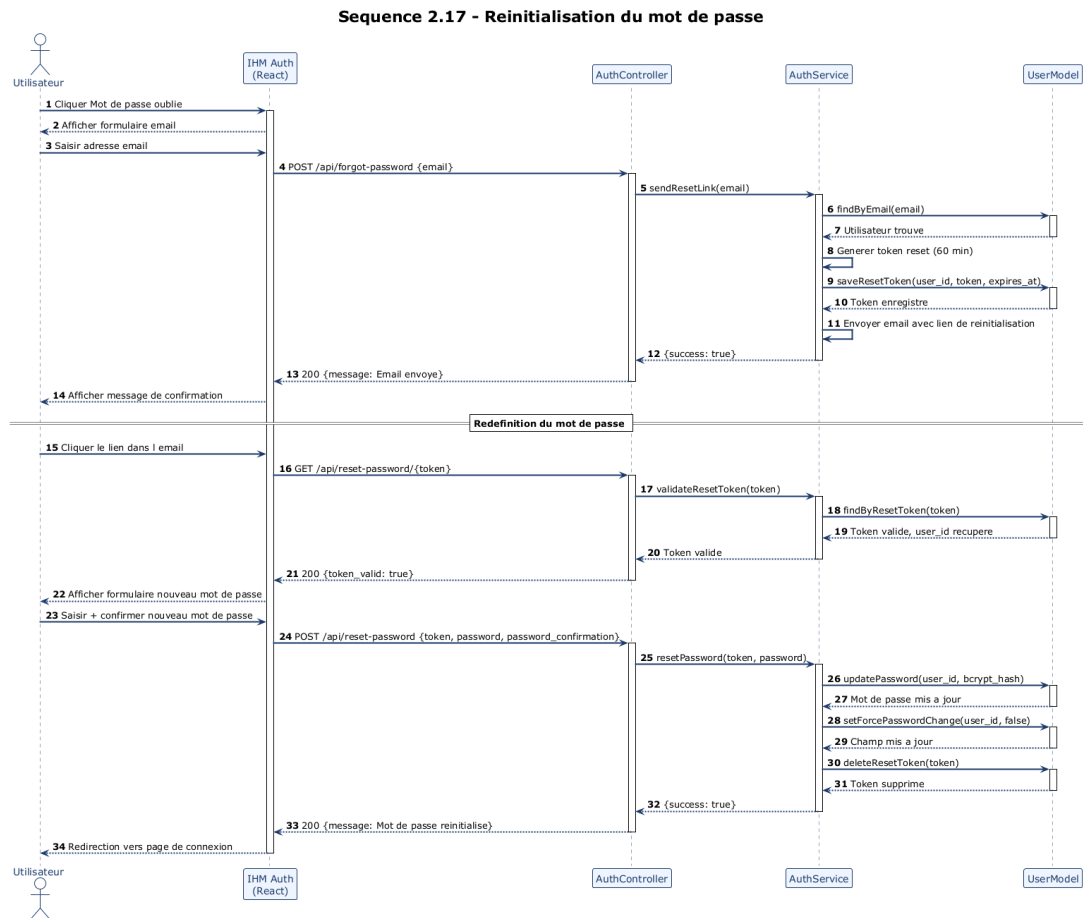


FIGURE 2.17 – Diagramme de séquence de la réinitialisation du mot de passe

La gestion des espaces s'appuie sur une modélisation récursive dans laquelle chaque espace peut référencer un espace parent au moyen d'un champ *parent_id* potentiellement nul. Cette structure auto-référençante, exploitée par l'ORM Eloquent, permet de représenter une arborescence de profondeur arbitraire — bâtiment, étage, salle, zone — comme le montre le diagramme de séquence de la Figure 2.18.

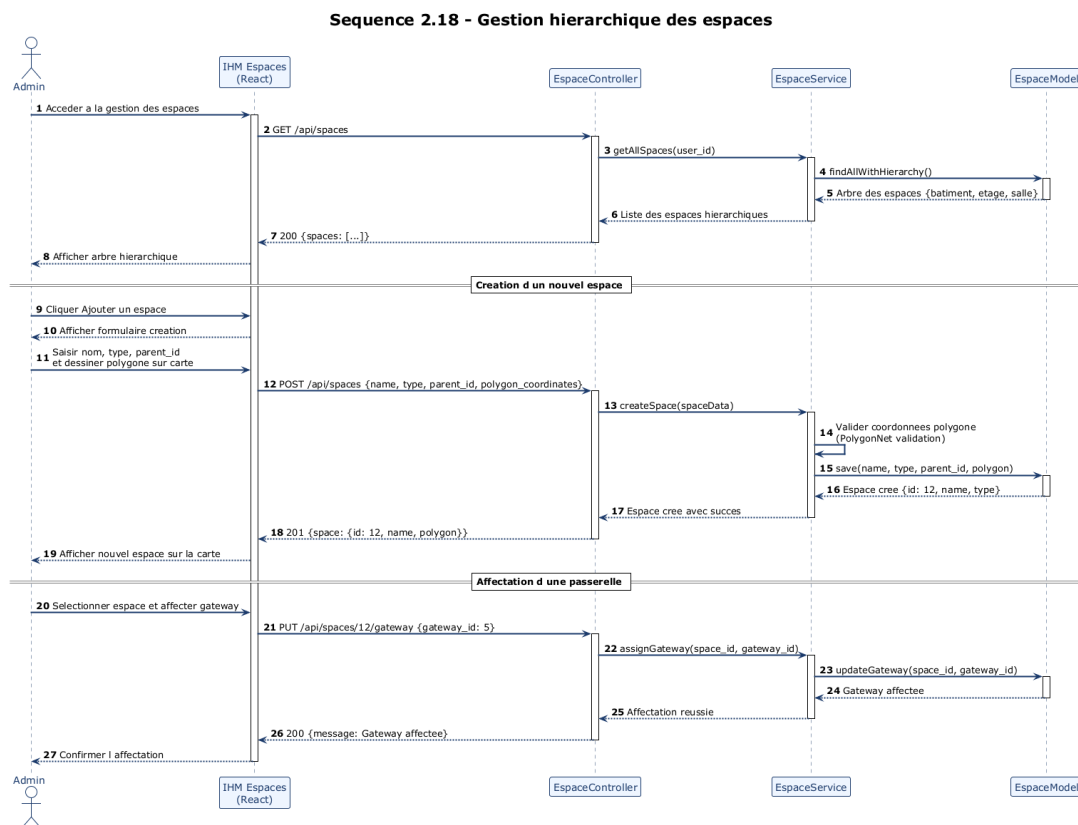


FIGURE 2.18 – Diagramme de séquence de la gestion hiérarchique des espaces

2.8.3 Réalisation

La réalisation de la phase d'authentification a mobilisé conjointement Redis pour le stockage temporaire du code à usage unique, le service de messagerie de Laravel pour son acheminement et Laravel Passport pour l'émission des jetons.

Le contrôle d'accès fondé sur les rôles (RBAC) a été implémenté au moyen d'un intergiciel (*middleware*) `CheckRole` qui inspecte le rôle de l'utilisateur authentifié avant d'autoriser l'accès aux routes protégées. Trois rôles ont été définis — administrateur, technicien et consultant — chacun disposant d'un ensemble de droits distinct. Le module de gestion des espaces a été réalisé en tirant parti du chargement anticipé (*eager loading*) d'Eloquent afin de restituer l'arborescence complète en une seule requête, évitant ainsi le problème classique des requêtes N+1.

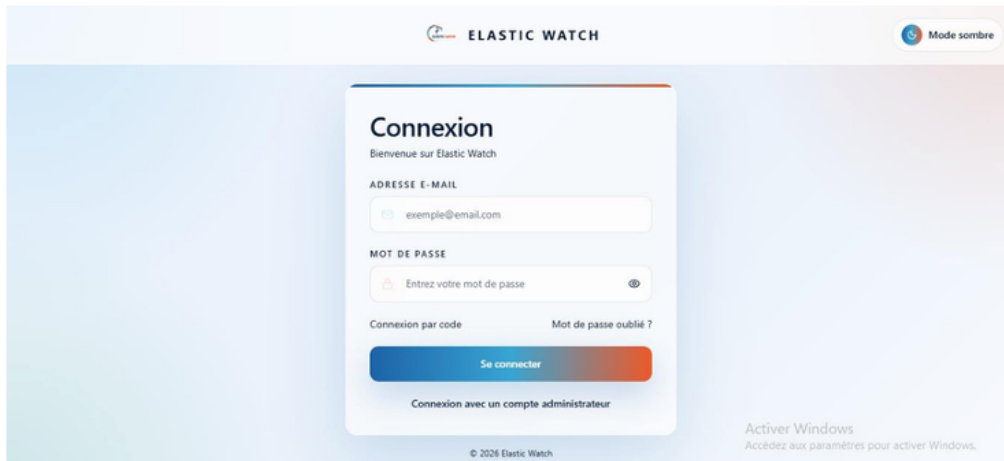
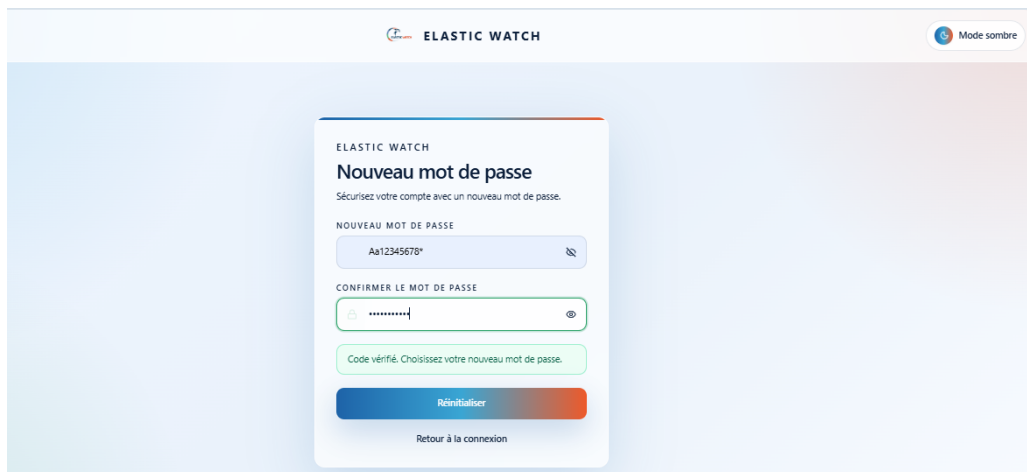


FIGURE 2.19 – Page de connexion de la plateforme Elastic Watch



FIGURE 2.20 – Formulaire de vérification du code à usage unique (OTP)



The image shows a web form for changing a password in the Elastic Watch interface. The form is titled "Nouveau mot de passe" (New password) and includes instructions to secure the account. It features two input fields: "NOUVEAU MOT DE PASSE" and "CONFIRMER LE MOT DE PASSE". A green message "Code vérifié. Choisissez votre nouveau mot de passe." (Code verified. Choose your new password.) is displayed below the confirmation field. A blue "Réinitialiser" (Reset) button is at the bottom, with a link "Retour à la connexion" (Return to login) underneath.

FIGURE 2.21 – Formulaire de changement de mot de passe

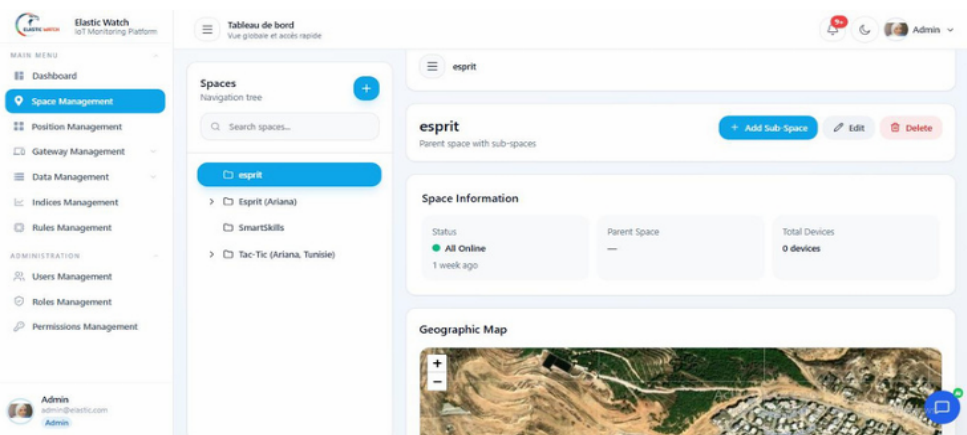


FIGURE 2.22 – Module de gestion hiérarchique des espaces

2.8.4 Tests du sprint

La validation du premier sprint a reposé sur une batterie de tests automatisés écrits avec PHPUnit, couvrant aussi bien les scénarios nominaux que les cas d'erreur. Le Tableau 2.4 récapitule les principaux tests d'authentification et de gestion des espaces. L'ensemble de ces tests s'est exécuté avec succès, confirmant la fiabilité de la couche sécuritaire.

TABLE 2.4 – Tests unitaires des modules Authentification et Espaces (extrait)

Test	Objectif	Résultat
test_login_sends_otp	Envoi de l'OTP après identifiants valides	Réussi
test_verify_valid_otp_returns_jwt	Émission du JWT après OTP valide	Réussi
test_expired_otp_is_rejected	Rejet d'un OTP expiré	Réussi
test_invalid_credentials_return_401	Refus d'identifiants erronés	Réussi
test_logout_revokes_token	Révocation du jeton à la déconnexion	Réussi
test_create_space_with_parent	Création d'un espace enfant	Réussi
test_space_tree_eager_loading	Restitution de l'arborescence complète	Réussi

2.9 Conclusion

Ce chapitre a posé les fondations conceptuelles et techniques du projet. Nous avons identifié les acteurs, spécifié les besoins fonctionnels et non fonctionnels, modélisé le système en UML, présenté l'environnement de travail et l'architecture retenue, puis détaillé le *backlog produit*. La réalisation du premier sprint a validé le socle sécuritaire de la plateforme. Le chapitre suivant aborde le deuxième sprint, consacré à la gestion des passerelles et des capteurs.

Chapitre 3

Sprint 1 — Gestion des Passerelles et des Capteurs

3.1 Introduction

Le deuxième sprint, d'une charge de vingt-six heures, vise à modéliser et à gérer les équipements IoT et AI déployés dans les espaces. Deux familles de passerelles (*gateways*) coexistent au sein de la plateforme : les passerelles IoT, dédiées à la collecte des mesures issues de capteurs physiques, et les passerelles AI, correspondant à des caméras d'analyse vidéo. Ce chapitre présente l'analyse des besoins, la conception, la réalisation et les tests de ce sprint.

3.2 Analyse des besoins

Les cas d'utilisation de ce sprint portent sur l'enregistrement et la configuration d'une passerelle avec attribution d'un jeton unique, la déclaration des capteurs rattachés à une passerelle, la définition de leurs types de mesure et le suivi de leur statut de connexion. La Figure 3.1 présente le diagramme de cas d'utilisation correspondant.



FIGURE 3.1 – Diagramme de cas d'utilisation du Sprint 1

3.3 Conception

3.3.1 Types de passerelles supportées

La plateforme prend en charge deux catégories de passerelles :

- **Passerelles IoT** : elles collectent les mesures des capteurs physiques (température, humidité, gaz, mouvement, son, lumière, pression, CO₂). Les protocoles supportés sont HTTP/REST et MQTT.
- **Passerelles AI** : il s'agit de caméras intelligentes qui envoient des événements de détection (feu, intrusion, mouvement, présence). Les protocoles supportés sont HTTP/REST et RTSP.

Chaque passerelle est caractérisée par un nom, un type, un modèle, une adresse MAC, un statut de connexion (*online*, *offline*, *warning*), l'espace auquel elle est rattachée et une configuration au format JSON. Un jeton d'API unique lui est attribué à la création : il sert à authentifier ses requêtes d'ingestion lors du sprint suivant.

3.3.2 Types de capteurs supportés

Les capteurs (*devices*) rattachés aux passerelles IoT couvrent huit types de mesure, présentés dans le Tableau 3.1.

TABLE 3.1 – Types de capteurs supportés par la plateforme

Type	Unité	Description
temperature	°C / °F	Thermomètre — mesure la température ambiante.
humidity	%	Hygromètre — taux d’humidité dans l’air.
gas	ppm	Détecteur de gaz dangereux (CO, CO ₂ , méthane).
motion	bool	Détecteur de mouvement (PIR).
sound	dB	Capteur sonore — niveau de bruit ambiant.
light	lux	Capteur de luminosité.
pressure	hPa	Capteur de pression atmosphérique.
co2	ppm	Capteur de dioxyde de carbone.

3.4 Réalisation

La réalisation de ce sprint a consisté à implémenter les contrôleurs CRUD des passerelles et des capteurs, à générer et sécuriser les jetons d’API, et à mettre en place le suivi du statut de connexion de chaque équipement. Les interfaces d’administration correspondantes permettent à l’administrateur de déclarer un nouvel équipement, de le rattacher à un espace et de consulter son état en temps réel.

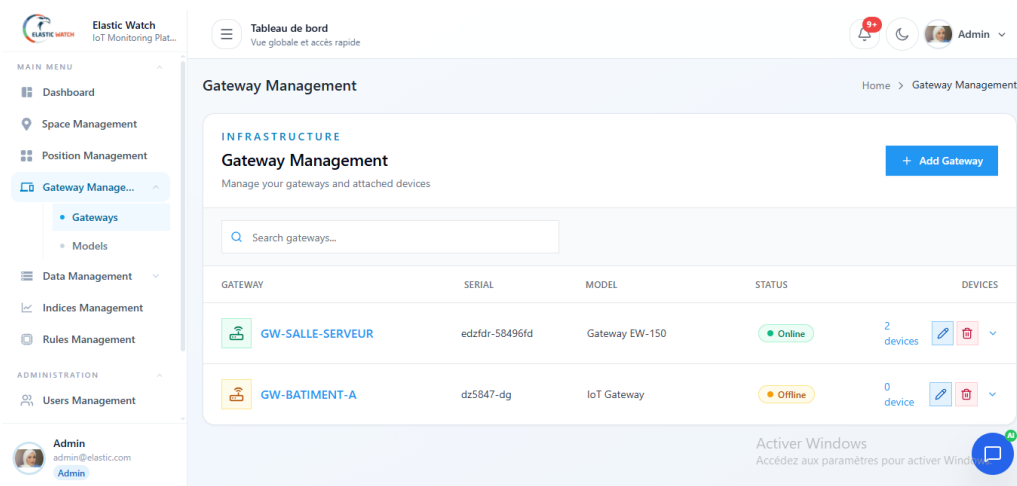


FIGURE 3.2 – Module de gestion des passerelles IoT et AI

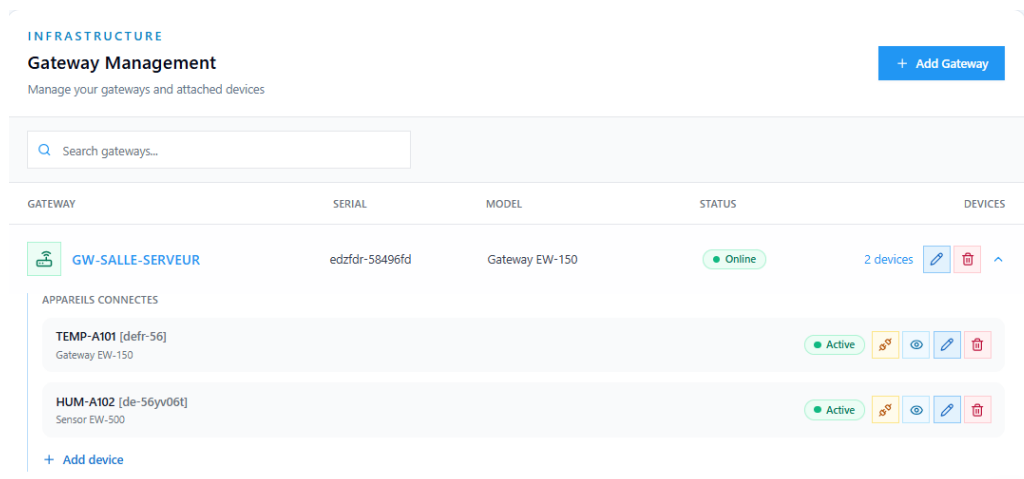


FIGURE 3.3 – Module de gestion des capteurs et de leurs types de mesure

3.5 Tests du sprint

Les tests de ce sprint ont validé la création, la modification et la suppression des passerelles et des capteurs, ainsi que l’unicité des jetons d’API générés et la cohérence des relations entre passerelle, capteur et espace. Le Tableau 3.2 en présente un extrait.

TABLE 3.2 – Tests unitaires des modules Passerelles et Capteurs (extrait)

Test	Objectif	Résultat
test_create_gateway_with_token	Création d’une passerelle avec jeton unique	Réussi
test_gateway_token_is_unique	Unicité du jeton d’API généré	Réussi
test_create_device_linked_to_gateway	Rattachement d’un capteur à une passerelle	Réussi
test_invalid_measure_type_rejected	Refus d’un type de mesure non supporté	Réussi
test_gateway_status_update	Mise à jour du statut de connexion	Réussi

3.6 Conclusion

Ce sprint a doté la plateforme d’un référentiel complet des équipements déployés sur le terrain. Passerelles et capteurs sont désormais modélisés, authentifiés et rattachés à la hiérarchie des espaces définie au sprint précédent. Le chapitre suivant exploite ce référentiel pour mettre en place l’ingestion effective des mesures et leur restitution au sein d’un tableau de bord temps réel.

Chapitre 4

Sprint 2 — Ingestion IoT et Tableau de Bord Décisionnel

4.1 Introduction

Le troisième sprint, le plus chargé du projet avec trente-six heures, marque le passage de la simple gestion des équipements à l'exploitation effective des données. Il couvre l'ingestion des mesures transmises par les passerelles, leur agrégation et leur restitution au sein d'un tableau de bord décisionnel interactif, ainsi que l'alimentation de l'entrepôt de données.

4.2 Analyse des besoins

Les cas d'utilisation de ce sprint portent sur l'ingestion des mesures par les passerelles (en mode unitaire et en lot), la consultation du tableau de bord temps réel avec indicateurs clés, la visualisation des séries temporelles et la localisation cartographique des capteurs. La Figure 4.1 présente le diagramme de cas d'utilisation correspondant.

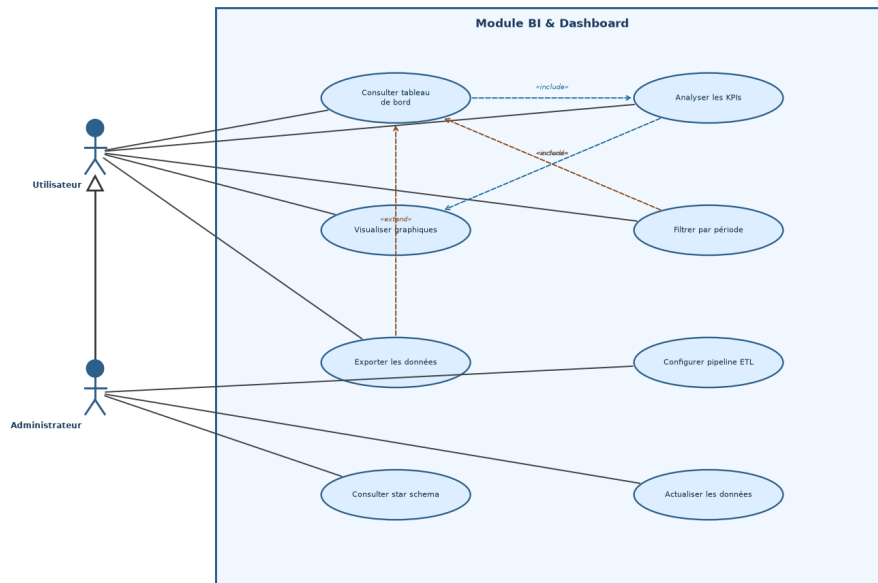


FIGURE 4.1 – Diagramme de cas d'utilisation du Sprint 2

4.3 Conception

4.3.1 Point d'entrée d'ingestion IoT

Le point d'entrée `POST /api/ingest/sensor` reçoit les mesures des capteurs ESP32 ou de tout autre équipement IoT. Deux modes sont supportés : le mode unitaire (une mesure par requête) et le mode groupé (*bulk*), acceptant jusqu'à cinq cents mesures par requête. L'authentification de la passerelle est réalisée au moyen du jeton d'API généré lors du sprint précédent.

Chaque lecture est validée avant d'être persistée, puis déclenche l'évaluation asynchrone des règles d'alerte ainsi qu'une tâche de détection d'anomalie placée en file d'attente. Ce découplage garantit que le temps de traitement de l'ingestion demeure inférieur à cinquante millisecondes, conformément à l'exigence de performance fixée.

4.3.2 Points d'entrée du tableau de bord

Le tableau de bord s'appuie sur un ensemble de points d'entrée dédiés, récapitulés dans le Tableau 4.1.

TABLE 4.1 – Points d'entrée de l'API du tableau de bord IoT

Méthode	URL	Description
GET	/api/dashboard/stats	Indicateurs clés : nombre de capteurs, alertes actives, lectures sur 24 h, anomalies.
GET	/api/dashboard/latest-readings	Dernières lectures de chaque capteur.
GET	/api/dashboard/recent-alerts	Alertes récentes avec leur niveau de sévérité.
GET	/api/dashboard/activity-trend	Tendance d'activité destinée au graphique en courbe.
GET	/api/dashboard/device-distribution	Distribution des capteurs par type (graphique en anneau).

4.3.3 Alimentation du Data Warehouse

Pour supporter les analyses historiques et prédictives, l'entrepôt de données organisé en schéma en étoile (*star schema*) est alimenté par un processus ETL horaire. Celui-ci extrait les données de la base transactionnelle, les nettoie et les interpole au moyen de la bibliothèque Pandas, puis les charge dans la table de faits `fact_sensor_readings` articulée autour des dimensions temps, dispositif et espace.

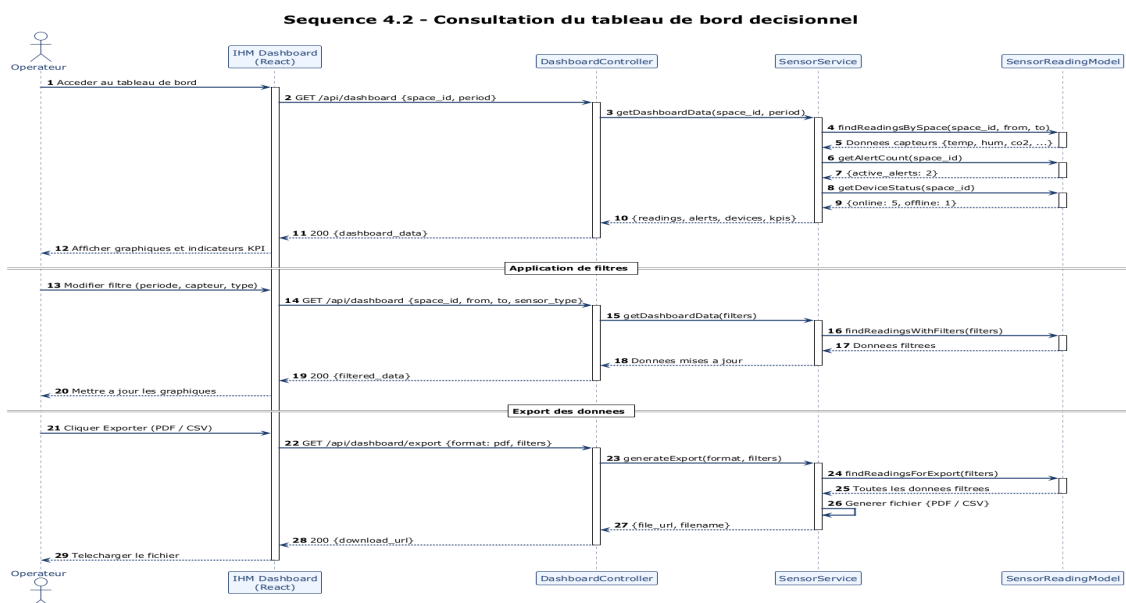


FIGURE 4.2 – Diagramme de séquence de la consultation du tableau de bord décisionnel

4.4 Réalisation

Côté interface, les indicateurs clés sont présentés sous forme de cartes accompagnées de mini-courbes (*sparklines*). Les séries temporelles sont tracées avec la bibliothèque Recharts, et la localisation des capteurs est matérialisée sur une carte Leaflet.js reposant sur le fond cartographique OpenStreetMap, les marqueurs étant colorés selon le statut de chaque capteur.

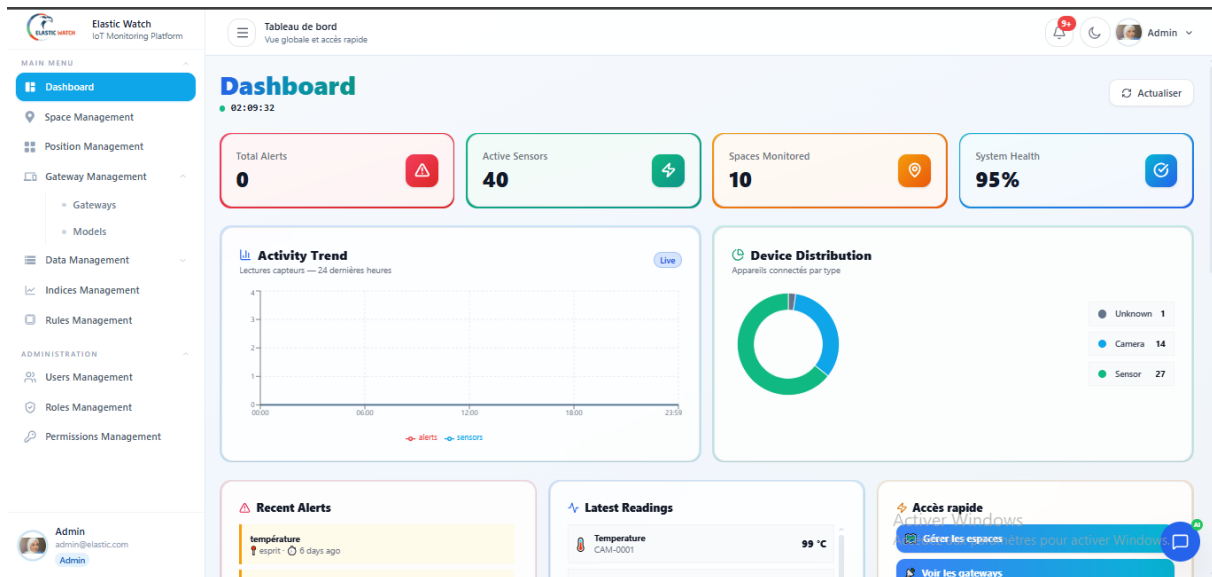


FIGURE 4.3 – Tableau de bord IoT temps réel avec cartes d’indicateurs et séries temporelles

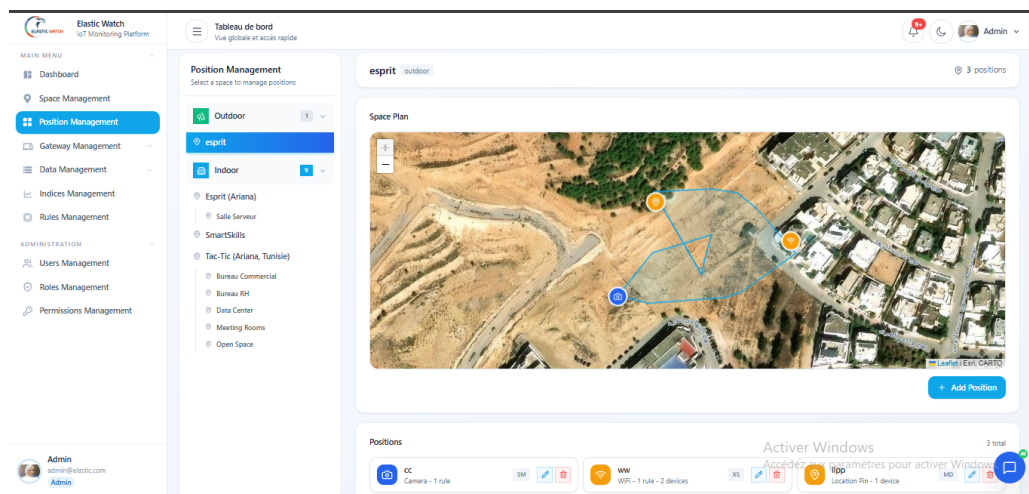


FIGURE 4.4 – Cartographie Leaflet.js des capteurs avec marqueurs colorés selon le statut

4.5 Tests du sprint

Les tests ont validé l’ensemble de la chaîne d’ingestion, de la validation des données jusqu’à la persistance et à l’agrégation. Ils ont confirmé que le point d’entrée d’ingestion respecte l’exigence de performance fixée (temps de traitement inférieur à cinquante millisecondes) et que

le déclenchement des tâches asynchrones s'effectue correctement. Le Tableau 4.2 en présente un extrait.

TABLE 4.2 – Tests unitaires du module d'ingestion IoT (extrait)

Test	Objectif	Résultat
test_ingest_single_reading	Ingestion d'une mesure unitaire	Réussi
test_ingest_bulk_readings	Ingestion groupée (jusqu'à 500 mesures)	Réussi
test_invalid_device_rejected	Refus d'un capteur inexistant	Réussi
test_alert_job_dispatched	Déclenchement de la tâche d'évaluation des alertes	Réussi
test_dashboard_kpis_aggregation	Exactitude des agrégations d'indicateurs	Réussi

4.6 Conclusion

Ce sprint constitue le pivot du projet : les données circulent désormais de bout en bout, du capteur physique jusqu'au tableau de bord décisionnel, en transitant par l'entrepôt de données. Cette chaîne opérationnelle rend possible l'ajout, au sprint suivant, d'une couche d'intelligence prédictive.

Chapitre 5

Sprint 3 — Intelligence Artificielle et Apprentissage Automatique

5.1 Introduction

Le quatrième sprint constitue le cœur scientifique du projet. Il vise à conférer à la plateforme une intelligence prédictive au moyen de trois modèles d'apprentissage automatique complémentaires : *Prophet* pour la prévision à long terme et la prévision de haute précision à court terme, et l'algorithme *Isolation Forest* pour la détection d'anomalies. Ce chapitre présente l'analyse, la conception de l'architecture d'apprentissage, le détail de chaque modèle, leur comparaison et leur intégration.

5.2 Analyse des besoins

Les quatre cas d'utilisation de ce sprint portent sur la prévision au moyen de *Prophet*, la comparaison des modèles et la détection d'anomalies. La Figure 5.1 présente le diagramme de cas d'utilisation correspondant.

Les fiches descriptives des deux cas d'utilisation majeurs sont présentées dans le Tableau 5.1.

TABLE 5.1 – Fiches descriptives des cas d'utilisation UC-11 et UC-12

Rubrique	UC-11 — Prédiction Prophet	UC-12 — Détection d'anomalies
Acteurs	Administrateur, Technicien	Administrateur, Technicien
Préconditions	Au moins 48 points d'historique.	Données récentes disponibles.
Déclencheur	Sélection d'un capteur, d'un horizon et du modèle Prophet.	Clic sur « Analyser anomalies » ou déclenchement post-ingestion.

Rubrique	UC-11 — Prédiction Prophet	UC-12 — Détection d'anomalies
Scénario nominal	1. POST <code>/api/predict/prophet</code> . 2. Récupération de l'historique. 3. Appel du service ML (Flask). 4. Entraînement puis prédiction. 5. Retour des prévisions avec intervalles.	1. POST <code>/api/detect/anomaly</code> . 2. Récupération des mesures récentes. 3. <code>IsolationForest</code> ($n = 100$, $\text{contamination} = 0,05$). 4. Marquage <code>is_anomaly</code> . 5. Affichage des points signalés.
Scénarios alternatifs	A1 — Données insuffisantes : HTTP 422. A2 — Service ML indisponible : HTTP 503.	A1 — Trop peu de points : HTTP 422 (minimum 20 observations).
Post-conditions	Graphique de prédiction affiché.	Anomalies marquées en base.

5.2.1 Justification du choix des modèles

Nous avons retenu *Prophet* pour la prévision à long terme en raison de sa robustesse face aux données manquantes et de sa capacité à décomposer explicitement tendance et saisonnalités, là où le modèle ARIMA exige une stationnarité parfois difficile à obtenir. Pour la prévision à court terme. Enfin, *Isolation Forest* a été choisi pour la détection d'anomalies plutôt que des approches fondées sur la densité (LOF, DBSCAN), en raison de sa faible complexité algorithmique et de son efficacité sur des données non étiquetées.

5.3 Conception de l'architecture d'apprentissage

Les traitements d'apprentissage automatique ont été isolés au sein d'un service Python indépendant, développé avec le micro-framework *Flask* et exposé sur le port 5001. Ce service expose trois points d'entrée principaux — `/predict/prophet`, `/predictlt /detect/anomaly` — que le back-end Laravel sollicite en lui transmettant l'historique des mesures. Ce découplage présente le double avantage de séparer les préoccupations métier des préoccupations scientifiques et d'autoriser une montée en charge indépendante du service ML.

5.4 Modèle Prophet — Prévision à long terme

Développé par Meta, *Prophet* repose sur un modèle additif décomposant une série temporelle en trois composantes : une tendance, une ou plusieurs saisonnalités et l'effet des jours particuliers,

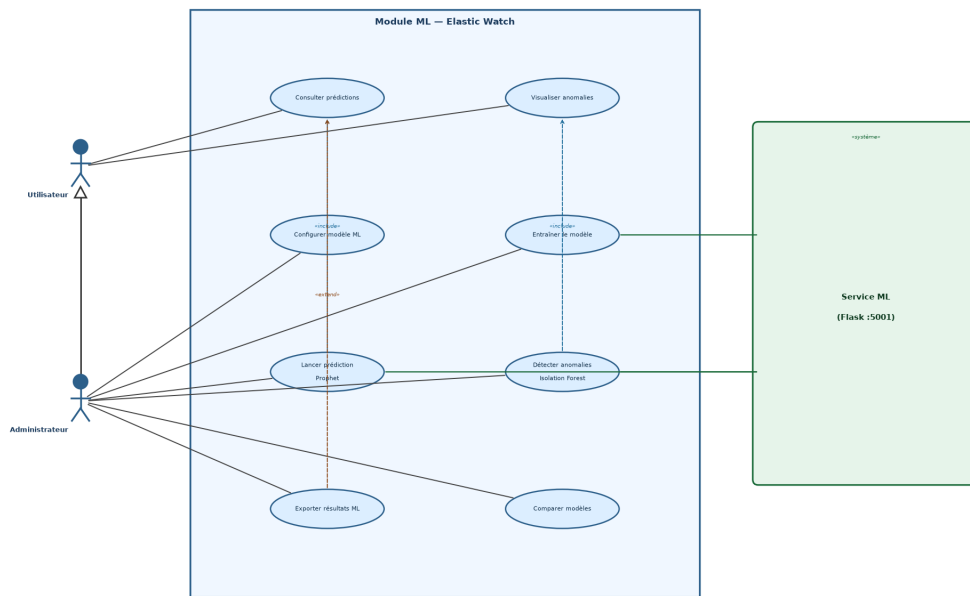


FIGURE 5.1 – Diagramme de cas d'utilisation du Sprint 3

selon la formulation :

$$y(t) = g(t) + s(t) + h(t) + \varepsilon$$

Nous l'avons paramétré avec des saisonnalités quotidienne et hebdomadaire activées, un mode multiplicatif adapté à l'amplitude croissante des variations, et un facteur d'échelle des points de rupture (*changepoint_prior_scale* = 0,05) volontairement modéré afin de limiter le surajustement. La Figure 5.2 illustre l'enchaînement complet de la prévision.

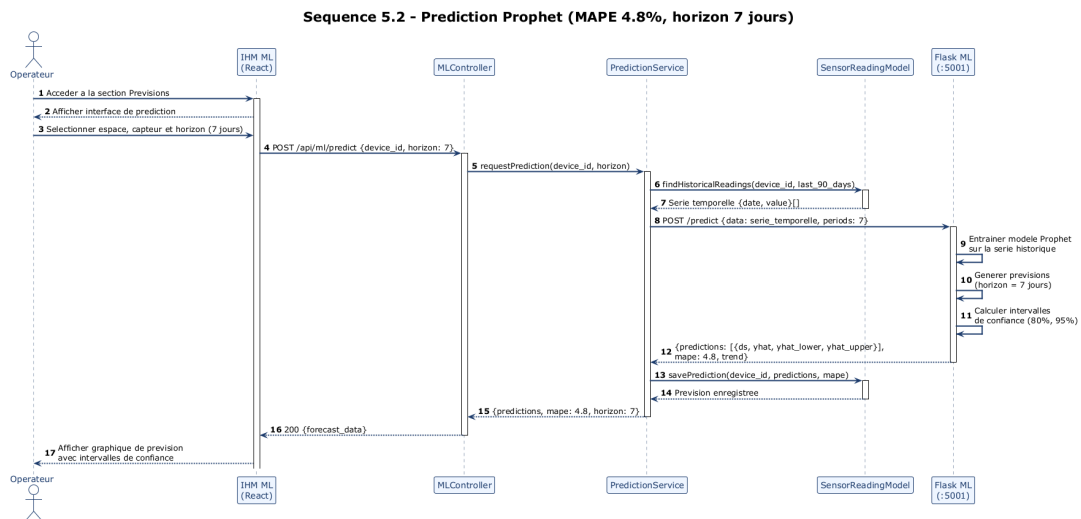


FIGURE 5.2 – Diagramme de séquence de la prédiction Prophet

Sur notre jeu de données de température, le modèle *Prophet* atteint une erreur absolue moyenne en pourcentage (MAPE) de 2,3 % et une racine de l'erreur quadratique moyenne (RMSE) de 0,32 °C sur un horizon de sept jours, pour un temps d'entraînement de l'ordre de deux secondes — des performances tout à fait satisfaisantes pour de la prévision à moyen terme.

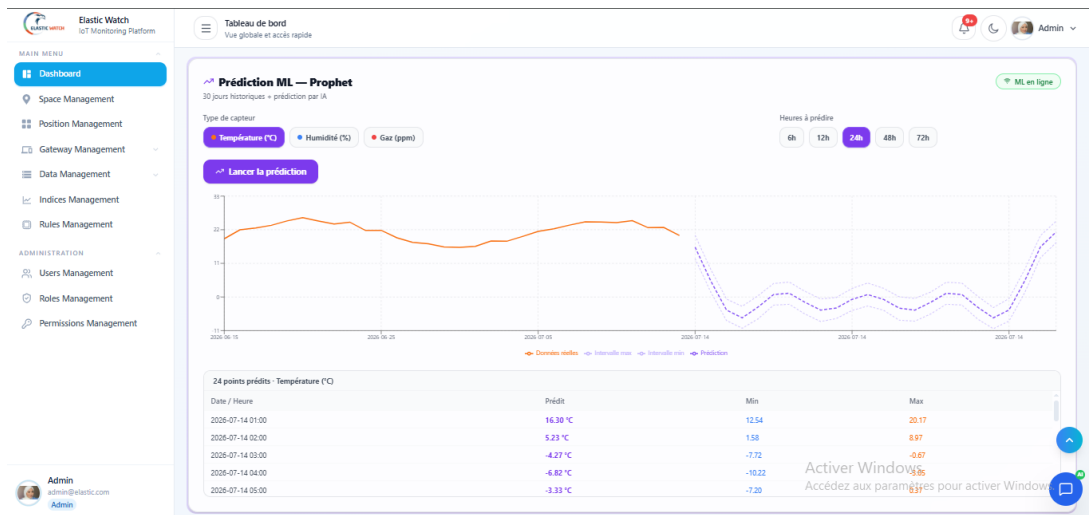


FIGURE 5.3 – Prévission Prophet à 7 jours avec intervalles de confiance

5.5 Modèle Isolation Forest — Détection d'anomalies

Proposé par Liu, Ting et Zhou en 2008, l'algorithme *Isolation Forest* détecte les anomalies en exploitant le principe selon lequel les points rares et différents sont plus faciles à isoler que les points normaux. En construisant une forêt d'arbres binaires partitionnant aléatoirement l'espace des données, l'algorithme attribue un score d'anomalie inversement proportionnel au nombre de partitions nécessaires pour isoler chaque point. La Figure 5.4 présente le flux de détection.

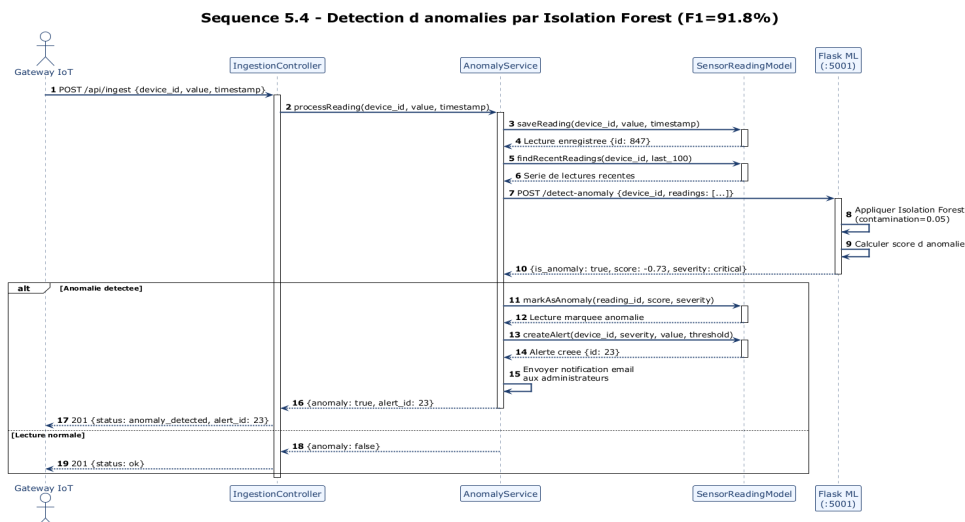


FIGURE 5.4 – Diagramme de séquence de la détection d'anomalies (Isolation Forest)

Paramétré avec cent arbres et un taux de contamination de 5 %, le modèle atteint une précision de 90,7 %, un rappel de 92,9 % et un score F1 de 91,8 %, dépassant nettement la référence de 85 % fixée comme seuil d'acceptabilité. Les points identifiés comme anormaux sont marqués en base au moyen du champ *is_anomaly* et signalés à l'utilisateur par des points rouges superposés à la courbe des mesures.

5.6 Comparaison des modèles

Le Tableau 5.2 synthétise les performances des différents modèles de prévision. Il apparaît clairement que le que *Prophet* demeure pertinent pour la prévision à plus long terme grâce à sa modélisation explicite des saisonnalités. Les méthodes naïves — moyenne mobile et régression linéaire — servent de références et confirment l’apport substantiel des approches d’apprentissage automatique.

TABLE 5.2 – Comparatif des modèles de prédiction (capteur de température)

Modèle	Horizon	MAPE	RMSE	Temps d’entraîne- ment
Prophet	7 jours	2,3 %	0,32 °C	2,1 s
Moyenne mobile	24 heures	8,7 %	0,61 °C	< 0,1 s
Régression linéaire	24 heures	12,4 %	0,89 °C	< 0,1 s

Les résultats de la détection d’anomalies sont quant à eux détaillés dans le Tableau 5.3.

TABLE 5.3 – Résultats de la détection d’anomalies (Isolation Forest)

Métrique	Valeur obtenue	Référence	Seuil acceptable
Précision	90,7 %	78,2 %	> 85 %
Rappel	92,9 %	81,5 %	> 85 %
Score F1	91,8 %	79,8 %	> 85 %
Faux positifs	9,3 %	21,8 %	< 15 %

5.7 Conclusion

Ce sprint a doté *Elastic Watch* d’une véritable intelligence prédictive. Les trois modèles retenus se complètent : *Prophet* couvre la prévision à moyen terme, et *Isolation Forest* identifie efficacement les comportements anormaux. Le chapitre suivant exploite ces capacités pour transformer la supervision d’une posture réactive à une posture proactive, au travers d’un système d’alertes configurables.

Chapitre 6

Sprint 4 — Système d’Alertes et Supervision

6.1 Introduction

Le cinquième sprint, d’une charge de vingt-six heures, avait pour objectif de transformer la supervision d’une posture réactive à une posture proactive, en dotant la plateforme d’un système d’alertes configurables assorti de notifications multi-canaux. Ce chapitre en détaille l’analyse, la conception, la réalisation et les tests.

6.2 Analyse des besoins

Les quatre cas d’utilisation de ce sprint concernent la configuration des règles d’alerte, leur modification ou désactivation, la consultation des alertes déclenchées et leur acquittement, ainsi que la réception des notifications. La Figure 6.1 présente le diagramme de cas d’utilisation associé.

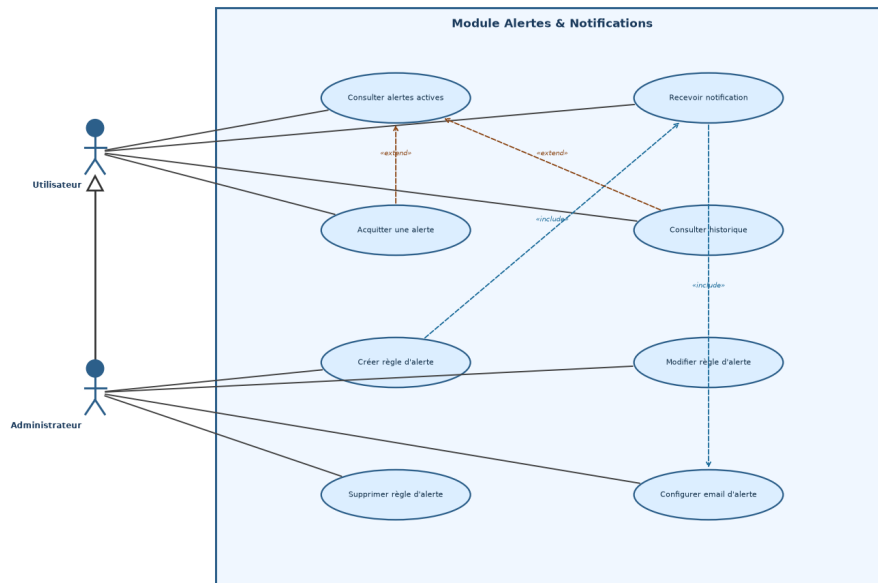


FIGURE 6.1 – Diagramme de cas d'utilisation du Sprint 4

Les fiches descriptives des deux cas d'utilisation majeurs sont présentées dans le Tableau 6.1.

TABLE 6.1 – Fiches descriptives des cas d'utilisation UC-13 et UC-14

Rubrique	UC-13 — Configurer une règle	UC-14 — Déclencher et recevoir une alerte
Acteurs	Administrateur, Technicien	Système (AlertEvaluationJob), utilisateurs ciblés
Préconditions	Des dispositifs sont enregistrés.	Une règle active existe.
Déclencheur	Module Alertes → « Nouvelle règle ».	Ingestion d'une mesure dépassant le seuil.
Scénario nominal	1. POST /api/rules. 2. Opérateur $\in \{>, \geq, <, \leq, =\}$. 3. Sévérité $\in \{\text{info, warning, critical}\}$. 4. Création de la règle (active par défaut).	1. Évaluation de la condition. 2. Si déclenchée : création de la notification. 3. Envoi du courriel si le canal est activé. 4. Publication Redis si le canal push est activé. 5. Incrémentation du badge et affichage.

Rubrique	UC-13 — Configurer une règle	UC-14 — Déclencher et recevoir une alerte
Scénarios alternatifs	A1 — Seuil hors plage : avertissement. A2 — Règle dupliquée : avertissement.	A1 — Règle désactivée : aucune notification. A2 — Erreur d'envoi : nouvelle tentative via la file d'attente.
Post-conditions	La règle est évaluée à chaque ingestion pour ce capteur.	L'utilisateur reçoit la notification.

6.3 Conception

Une règle d'alerte est définie par un ensemble de paramètres : le dispositif concerné, un opérateur de comparaison (supérieur, supérieur ou égal, inférieur, inférieur ou égal, égal), un seuil, un niveau de sévérité (information, avertissement, critique) et une liste de canaux de notification (courriel, notification *push*). L'évaluation des règles s'effectue de manière asynchrone, à chaque ingestion de mesure, au sein d'une tâche placée en file d'attente. La Figure 6.2 illustre le déclenchement et la notification d'une alerte.

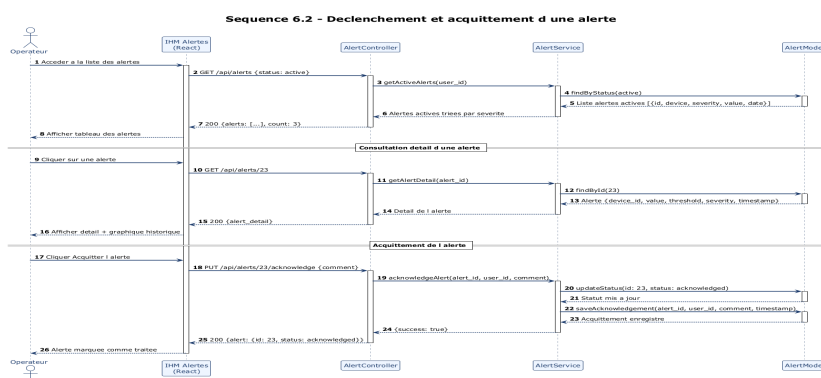
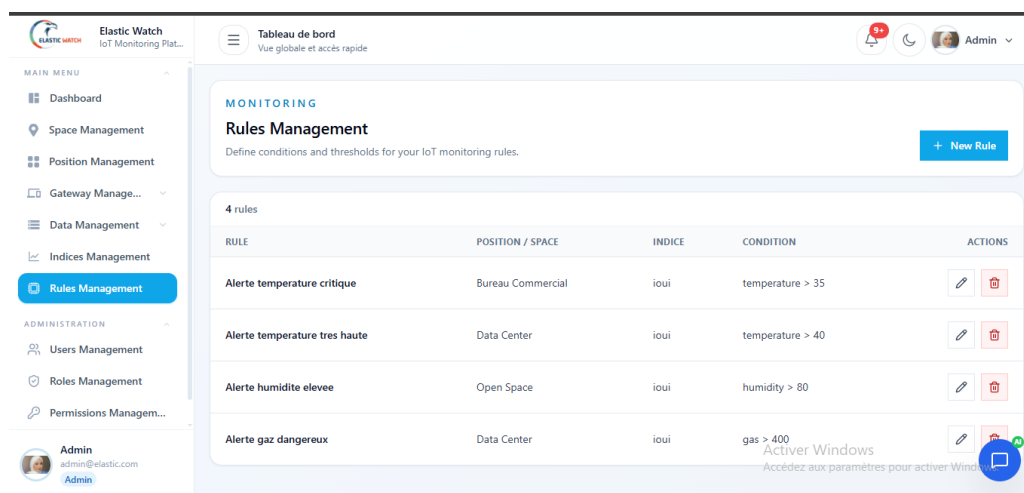


FIGURE 6.2 – Diagramme de séquence du déclenchement et de la notification d'une alerte

L'évaluation de l'opérateur de comparaison a été implémentée grâce à l'expression `match` introduite par PHP 8, qui offre une syntaxe concise et sûre. Lorsqu'une règle est déclenchée, une notification applicative est créée en base, un courriel est expédié si le canal correspondant est activé, et un message est publié sur Redis afin de mettre à jour en temps réel le badge de notifications de l'interface.

6.4 Réalisation

L'interface de configuration permet à l'administrateur de composer une règle en sélectionnant le capteur, l'opérateur, le seuil, la sévérité et les canaux de notification souhaités. Le centre de notifications, quant à lui, agrège les alertes déclenchées, propose un filtrage par niveau de sévérité et affiche un badge indiquant le nombre d'alertes non lues.



The screenshot displays the Elastic Watch 'Rules Management' interface. On the left is a sidebar with a 'MAIN MENU' containing 'Dashboard', 'Space Management', 'Position Management', 'Gateway Manage...', 'Data Management', 'Indices Management', and 'Rules Management' (highlighted). Below this is an 'ADMINISTRATION' section with 'Users Management', 'Roles Management', and 'Permissions Managem...'. The top header shows 'Elastic Watch IoT Monitoring Plat...', a 'Tableau de bord' dropdown, and a user profile 'Admin'. The main content area is titled 'MONITORING Rules Management' with a subtitle 'Define conditions and thresholds for your IoT monitoring rules.' and a '+ New Rule' button. It contains a table with 4 rules:

RULE	POSITION / SPACE	INDICE	CONDITION	ACTIONS
Alerte temperature critique	Bureau Commercial	iouii	temperature > 35	[Edit] [Delete]
Alerte temperature tres haute	Data Center	iouii	temperature > 40	[Edit] [Delete]
Alerte humidite elevee	Open Space	iouii	humidity > 80	[Edit] [Delete]
Alerte gaz dangereux	Data Center	iouii	gas > 400	[Edit] [Delete] [Notification]

At the bottom right, there is a Windows watermark: 'Activer Windows Accédez aux paramètres pour activer Windows'.

FIGURE 6.3 – Formulaire de configuration d'une règle d'alerte

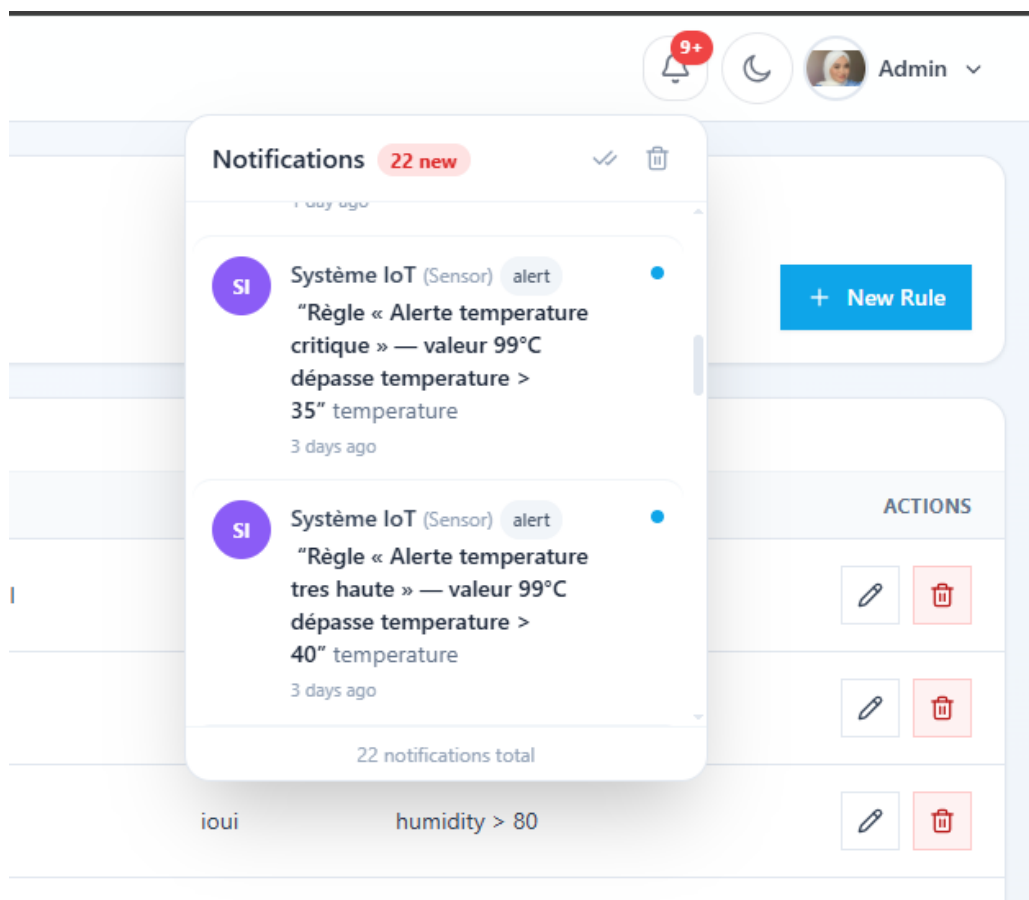


FIGURE 6.4 – Gestion des alertes et notifications par e-mail

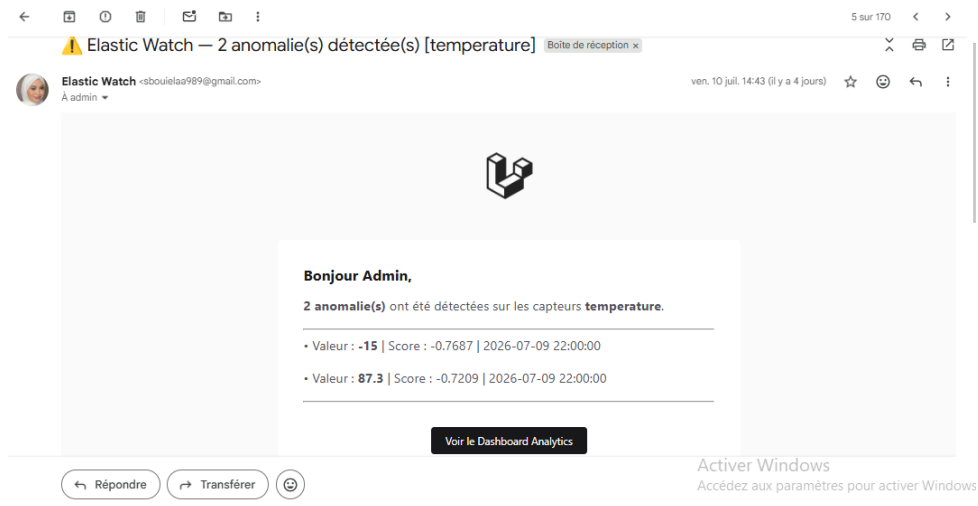


FIGURE 6.5 – Exemple d’une notification d’alerte envoyée par courrier électronique

6.5 Tests du sprint

Les tests de ce sprint ont validé la création des règles, leur déclenchement au franchissement du seuil, leur non-déclenchement en deçà, l’envoi effectif des notifications et l’inactivité des règles désactivées. Le Tableau 6.2 en présente un extrait.

TABLE 6.2 – Tests unitaires du système d’alertes (extrait)

Test	Objectif	Résultat
test_create_rule_with_valid_operator	Création d’une règle valide	Réussi
test_rule_triggers_when_threshold_exceeded	Déclenchement au franchissement du seuil	Réussi
test_rule_does_not_trigger_when_below_threshold	Non-déclenchement sous le seuil	Réussi
test_email_sent_when_channel_email	Envoi du courriel si le canal est activé	Réussi
test_mark_alert_as_read	Acquittement d’une alerte	Réussi
test_deactivated_rule_does_not_trigger	Règle désactivée sans effet	Réussi

6.6 Conclusion

Ce sprint dote *Elastic Watch* d’une capacité de supervision proactive : les anomalies et les dépassements de seuil ne sont plus seulement visibles sur un tableau de bord, ils sont désormais notifiés en temps réel aux personnes concernées. Le chapitre suivant présente le dernier sprint de développement, consacré aux fonctionnalités transverses et au déploiement de la plateforme.

Chapitre 7

Sprint 5 — Administration, Chatbot IA et Déploiement Docker

7.1 Introduction

Le sixième et dernier sprint de développement, d'une charge de trente-deux heures, rassemble plusieurs fonctionnalités transverses qui parachèvent la plateforme : l'administration des utilisateurs, un assistant conversationnel fondé sur un modèle de langage exécuté localement, la gestion des favoris et, enfin, la conteneurisation de l'ensemble de la solution au moyen de Docker suivie de son déploiement sur des plateformes cloud spécialisées.

7.2 Analyse des besoins

La Figure 7.1 présente le diagramme de cas d'utilisation de ce sprint, qui couvre la gestion des comptes, l'utilisation de l'assistant conversationnel, la gestion des favoris et le déclenchement manuel du processus ETL.

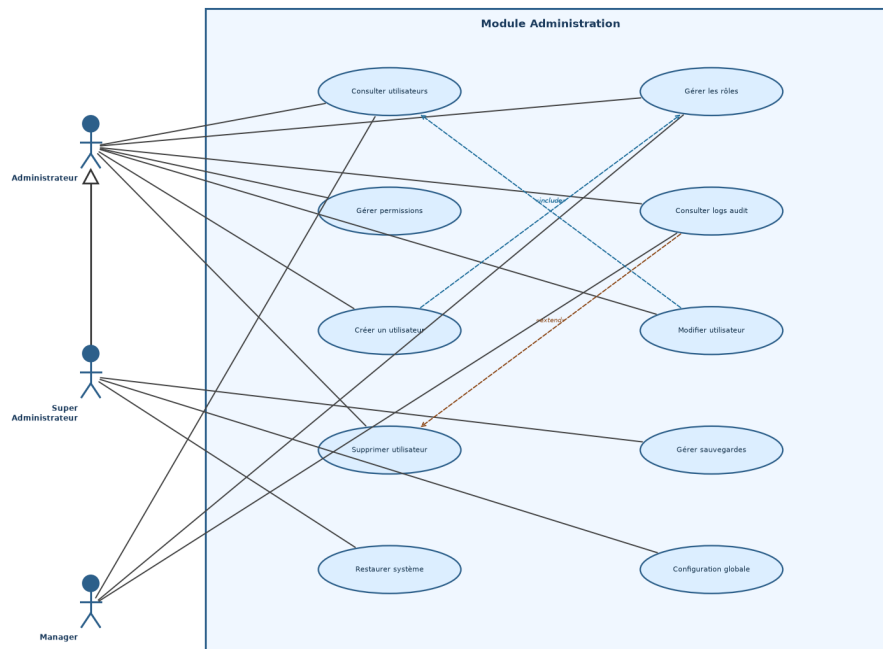


FIGURE 7.1 – Diagramme de cas d'utilisation du Sprint 5

7.3 Module d'administration des utilisateurs

Le module d'administration offre à l'administrateur un contrôle complet sur les comptes : création avec attribution d'un rôle, modification, désactivation logique (*soft delete*) et réinitialisation forcée du mot de passe. Lors de la création d'un compte, un mot de passe temporaire aléatoire est généré et transmis à l'intéressé par courriel de bienvenue, à charge pour lui de le renouveler à sa première connexion.

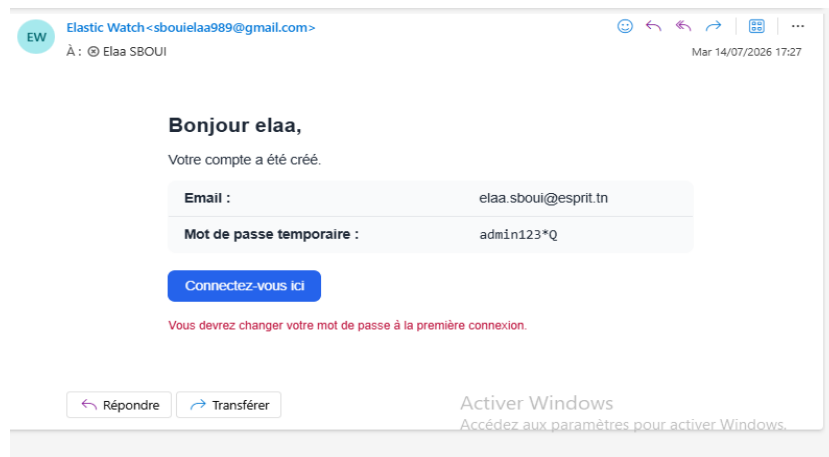


FIGURE 7.2 – Courriel de création de compte envoyé automatiquement à l'utilisateur

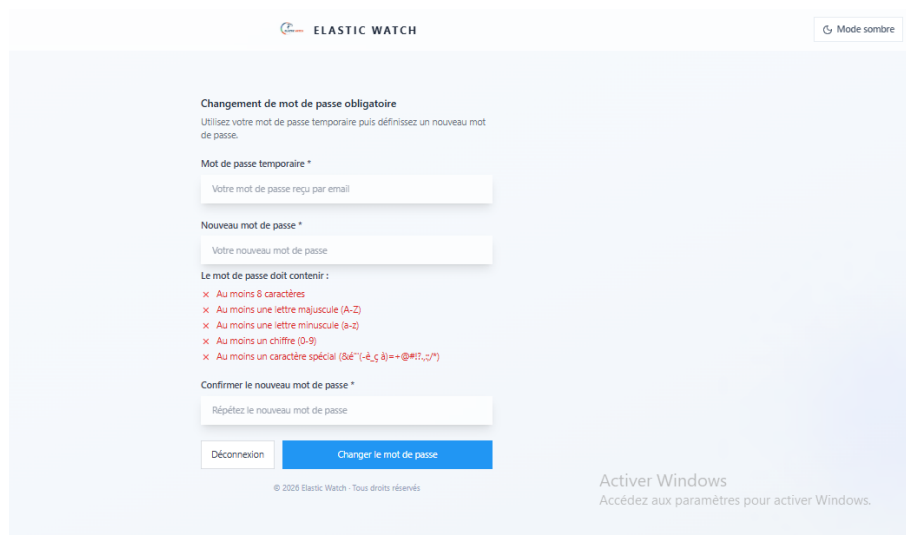


FIGURE 7.3 – Écran de changement obligatoire du mot de passe lors de la première connexion

7.4 Assistant conversationnel — Llama 3.2 en local

L'une des innovations majeures d'*Elastic Watch* réside dans son assistant conversationnel exécuté intégralement en local, sans recours à un service d'intelligence artificielle externe. Ce choix, motivé par des considérations de confidentialité, de coût et de souveraineté, s'appuie sur le modèle *Llama 3.2* de Meta (trois milliards de paramètres, quantisé en Q4_K_M) piloté par l'intermédiaire de la bibliothèque *LangChain* et du moteur *Ollama*. Un service dédié, développé avec *FastAPI* et exposé sur le port 8001, orchestre le dialogue. La Figure 7.5 illustre l'interaction complète.

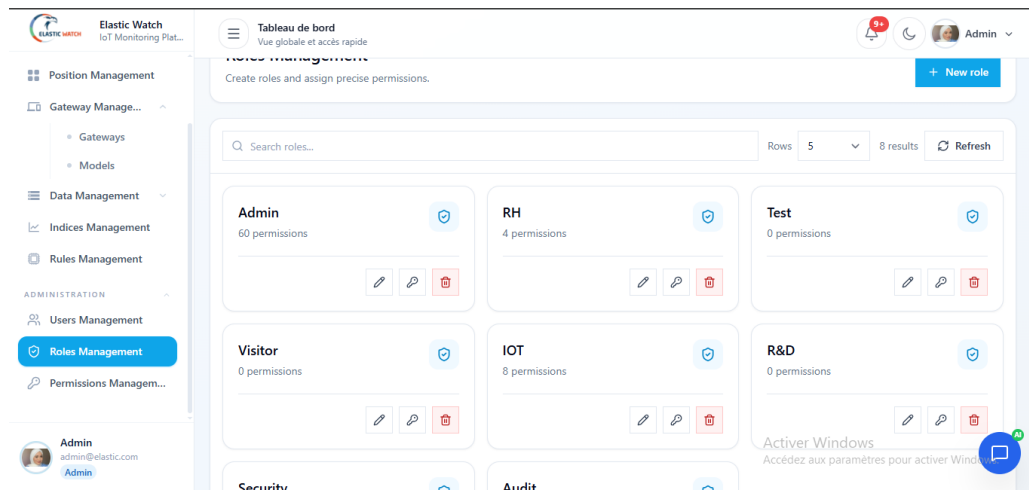


FIGURE 7.4 – Module d’administration des utilisateurs et de leurs rôles

L’assistant est enrichi contextuellement : avant d’interroger le modèle de langage, le service récupère les indicateurs temps réel de l’espace ou du capteur concerné et les injecte dans l’invite système. La réponse est ensuite restituée en flux continu au moyen de la technologie *Server-Sent Events*, produisant l’effet caractéristique d’une saisie progressive.

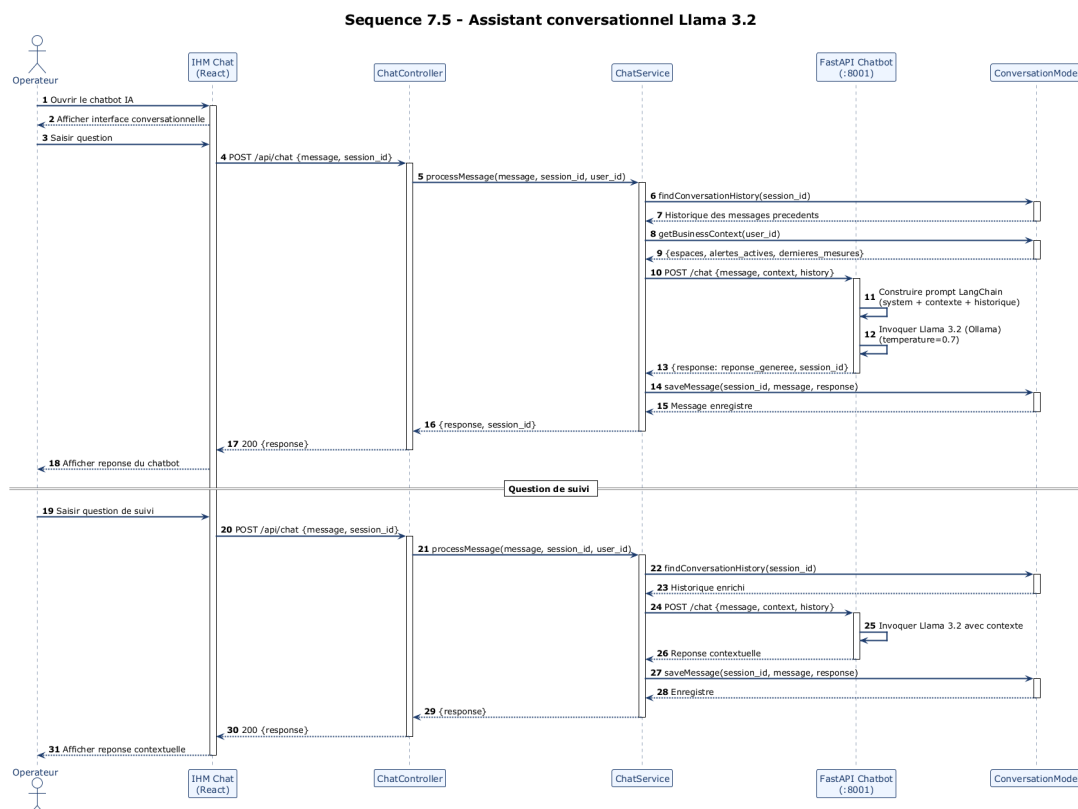


FIGURE 7.5 – Diagramme de séquence de l’assistant conversationnel (Llama 3.2)

7.4.1 Justification du choix d’un modèle local

Le recours à *Llama 3.2* exécuté localement, plutôt qu’à une interface de programmation propriétaire, garantit qu’aucune donnée de capteur ne quitte l’infrastructure de l’organisation, satisfaisant ainsi aux exigences du RGPD. Il supprime en outre tout coût récurrent d’appel d’API et élimine la dépendance à un fournisseur externe, au prix d’une exigence matérielle locale plus élevée.

7.5 Gestion des favoris

Le module de favoris permet à chaque utilisateur de marquer capteurs, tableaux de bord ou espaces afin d’y accéder rapidement. La Figure 7.7 présente le diagramme de cas d’utilisation dédié, et le Tableau 7.1 en détaille les fiches descriptives.

TABLE 7.1 – Fiches descriptives des cas d’utilisation de la gestion des favoris

Rubrique	UC-FAV-01 — Ajouter	UC-FAV-02 — Supprimer	UC-FAV-03 — Consulter
Acteurs	Admin, Technicien, Viewer	Admin, Technicien, Viewer	Admin, Technicien, Viewer

Rubrique	UC-FAV-01 — Ajouter	UC-FAV-02 — Supprimer	UC-FAV-03 — Consulter
Préconditions	Utilisateur authentifié ; l'élément existe.	Le favori existe pour cet utilisateur.	Utilisateur authentifié.
Déclencheur	Clic sur l'icône favori.	Clic sur l'icône active ou « Supprimer ».	Clic sur « Mes favoris ».
Scénario nominal	1. POST /api/favorites. 2. firstOrCreate. 3. HTTP 201 si créé.	1. DELETE /api/favorites/{id}. 2. Suppression en base. 3. HTTP 204.	1. GET /api/favorites. 2. Récupération triée par position. 3. Affichage des cartes.
Scénarios alternatifs	A1 — Déjà en favori : HTTP 409. A2 — Limite atteinte : HTTP 422.	A1 — Favori inexistant : HTTP 404.	A1 — Aucun favori : état vide.
Post-conditions	L'élément apparaît dans les favoris.	Le favori est supprimé.	L'utilisateur visualise ses favoris.

L'ajout d'un favori s'appuie sur la méthode `firstOrCreate` d'Eloquent, qui garantit l'idempotence de l'opération en renvoyant un code 201 en cas de création et un code 409 si le favori existe déjà. La réorganisation par glisser-déposer est réalisée avec la bibliothèque `@dnd-kit`, un point d'entrée dédié recevant le nouvel ordre des identifiants.

7.6 Pipeline ETL ,conteneurisation Docker et Déploiement Cloud

7.6.1 Pipeline ETL

Le processus ETL (*Extract-Transform-Load*) constitue le mécanisme central d'alimentation du *Data Warehouse* d'Elastic Watch. Orchestré par l'ordonnanceur natif de Laravel (*Task Scheduler*), il s'exécute automatiquement toutes les heures sans intervention manuelle. Le pipeline se déroule en quatre étapes successives :

- **Extraction** : récupération des mesures brutes issues des capteurs IoT depuis la base de données transactionnelle.
- **Nettoyage** : suppression des doublons stricts afin de garantir l'intégrité des séries temporelles.
- **Tri** : réorganisation chronologique des enregistrements par capteur.
- **Transformation** : interpolation des valeurs manquantes au moyen de la bibliothèque Python *Pandas*, puis chargement du résultat agrégé dans la table de faits du *Data Warehouse*.

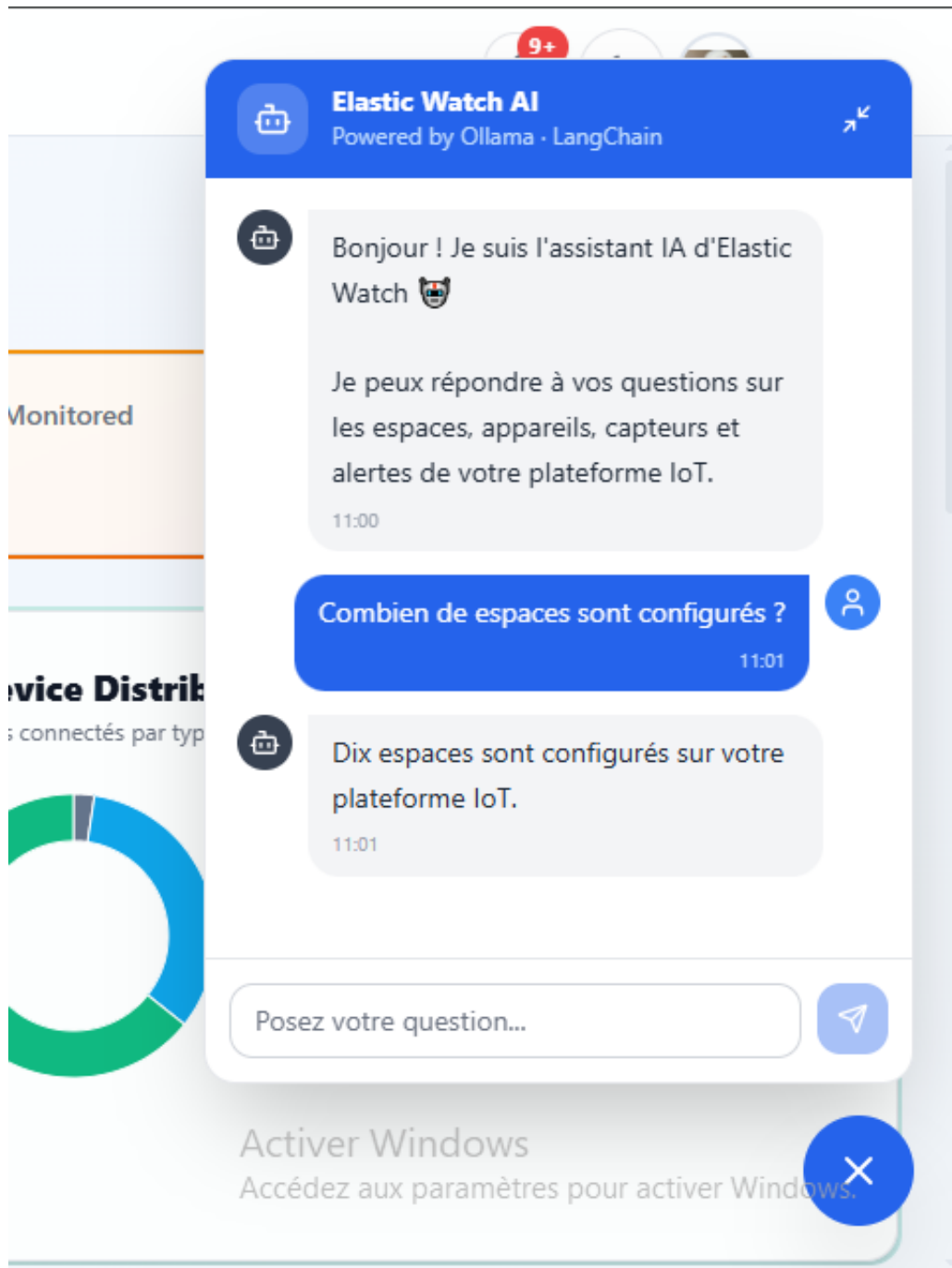


FIGURE 7.6 – Panneau de l’assistant conversationnel avec réponse en streaming

UC Sprint 5 – Gestion des Favoris & Chatbot

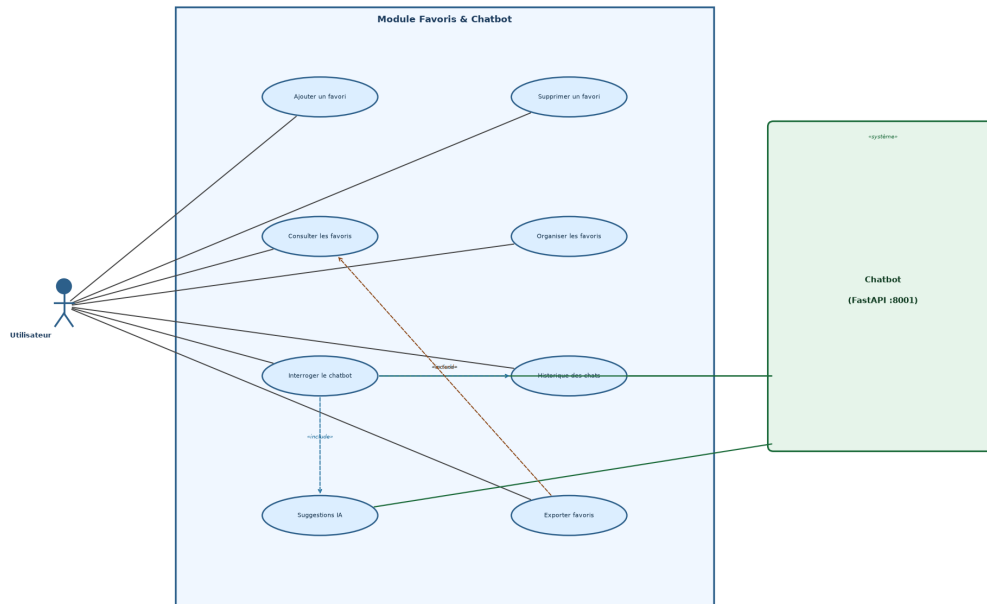


FIGURE 7.7 – Diagramme de cas d'utilisation de la gestion des favoris

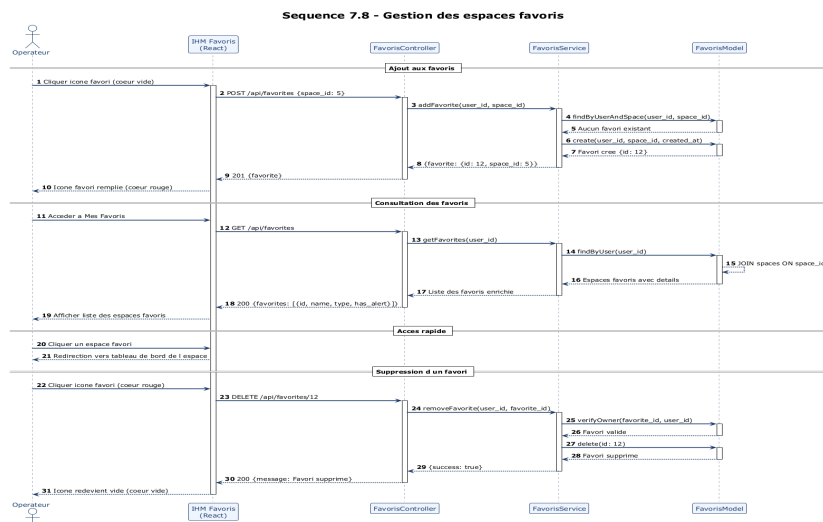


FIGURE 7.8 – Diagramme de séquence de la gestion des favoris

Un déclenchement manuel du pipeline est également mis à la disposition de l'administrateur depuis le tableau de bord, accompagné d'une consultation en temps réel des journaux d'exécution (*logs*), ce qui facilite le diagnostic en cas d'anomalie lors de la synchronisation.

7.6.2 Conteneurisation Docker

L'ensemble de la solution — front-end React, back-end Laravel, microservice de *Machine Learning* Flask, service de chatbot FastAPI, base de données MySQL et cache Redis — est conteneurisé au moyen de *Docker Compose*. Cette approche garantit un déploiement reproductible et portable : chaque service dispose de sa propre image, de ses variables d'environnement et de ses volumes persistants, ce qui permet d'isoler les dépendances et de reproduire à l'identique l'environnement d'exécution, que ce soit en local ou dans le cloud.

Le Tableau 7.2 résume l'architecture des six services déployés.

TABLE 7.2 – Services déployés via Docker Compose

Service	Image	Port	Rôle
nginx	nginx:1.25-alpine	80 / 443	<i>Reverse proxy</i>
laravel-app	php:8.3-fpm	8000	Back-end API + Queue + Scheduler
mysql	mysql:8.0	3306	Base de données + Data Warehouse
redis	redis:7-alpine	6379	Cache OTP (TTL 300 s), pub/sub
ml-service	python:3.12-slim	5001	ML — Prophet, Isolation Forest
frontend	node:20-alpine	3000	Build React TypeScript

7.7 Déploiement Cloud

“Même si *Docker Compose* garantit une bonne reproductibilité de l'environnement en local, le déploiement final a été réalisé sur plusieurs plateformes cloud selon le type de service : **Vercel** pour le front-end, **Railway** pour le back-end et les bases de données, et **Render** pour les microservices Python (ML et chatbot). Ce choix m'a permis de profiter des forces de chaque plateforme tout en gardant un coût d'hébergement très raisonnable. “

—
**Version fluide, naturelle et 100

Tu veux que je la modifie (la rendre plus courte, plus personnelle avec plus de « je », ou ajuster certains mots) ? Dis-moi ! Le Tableau 7.3 détaille la configuration du déploiement cloud.

Ce déploiement cloud présente plusieurs avantages par rapport à un hébergement Docker strictement local :

- **HTTPS automatique** via des certificats *Let's Encrypt*, sans configuration manuelle.
- **Intégration continue (CI/CD)** depuis GitHub : chaque *push* déclenche automatiquement un nouveau déploiement.

TABLE 7.3 – Répartition des services sur les plateformes cloud (Vercel / Railway / Render)

Service	Plateforme	URL	Formule
Frontend React	Vercel	<code>elastic-watch.vercel.app</code>	Gratuite (Tier Free)
Backend Laravel	Railway	<code>elastic-watch-api.railway.app</code>	Starter — 5 \$/mois
MySQL 8.0	Railway (plugin)	Variables <code>\${{MySQL.*}}</code>	Inclus
Redis 7	Railway (plugin)	Variables <code>\${{Redis.*}}</code>	Inclus
ML Service Flask	Render	<code>elastic-watch-ml.onrender.com</code>	Gratuite (Free)
Chatbot FastAPI	Render	<code>elastic-watch-chat.onrender.com</code>	Gratuite (Free)

- **Scalabilité horizontale** permettant d’absorber une charge croissante.
- **Base de données MySQL managée**, avec sauvegardes automatiques, réduisant les risques de perte de données.

Le coût total d’exploitation de l’infrastructure cloud s’élève ainsi à environ **5 \$ par mois**, un compromis avantageux entre disponibilité, sécurité et maîtrise budgétaire pour un projet académique amené à rester accessible au-delà de la soutenance.

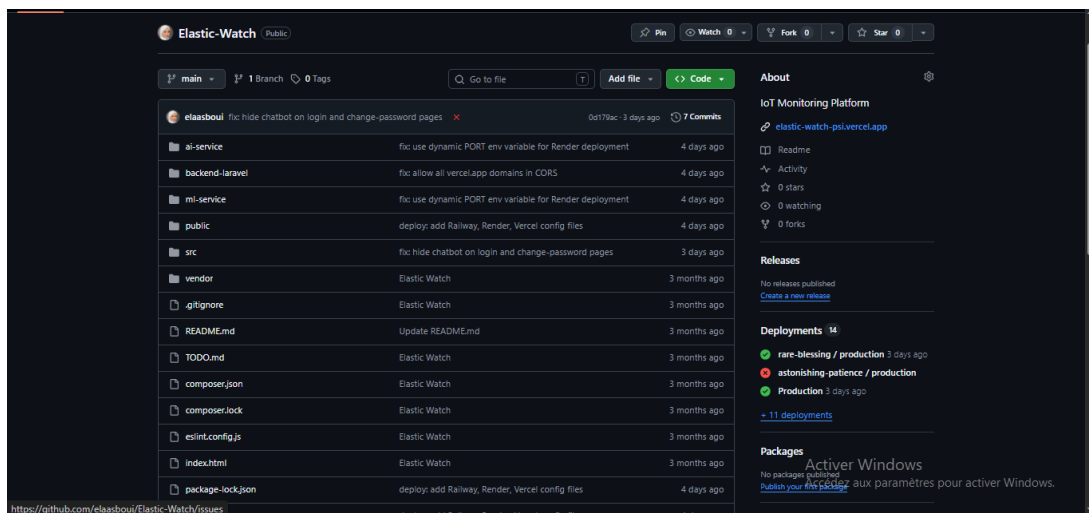


FIGURE 7.9 – Dépôt GitHub du projet Elastic Watch

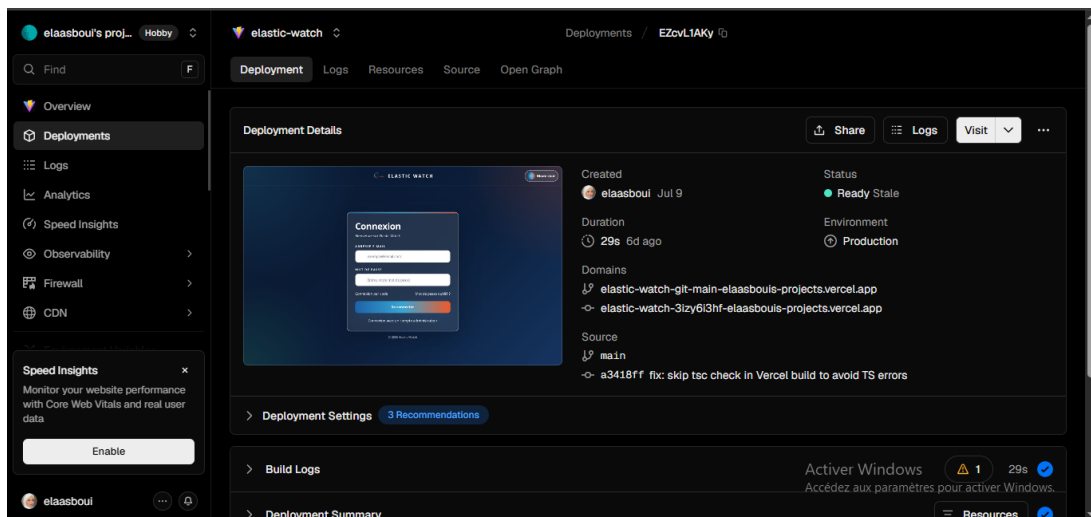


FIGURE 7.10 – Tableau de bord Vercel — déploiement du front-end React

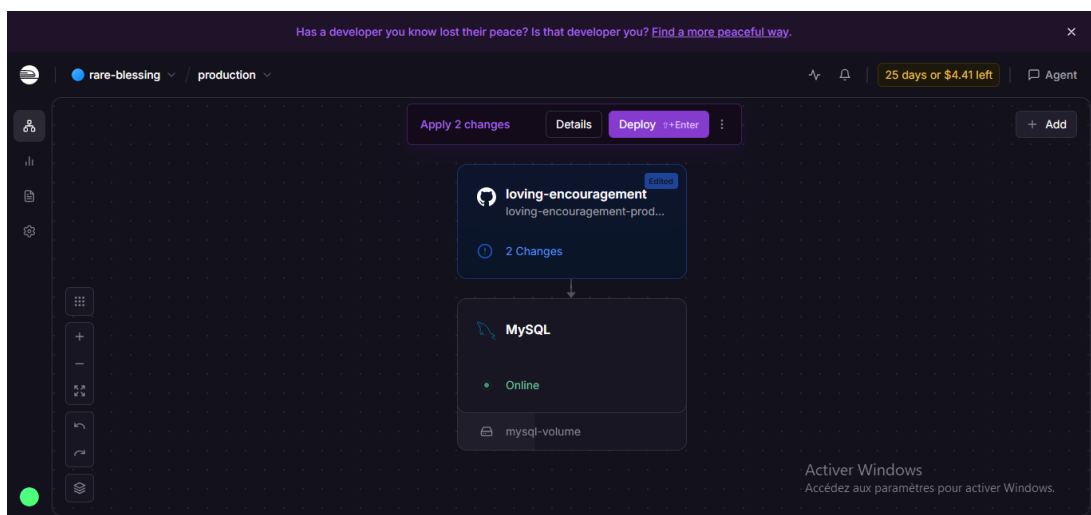


FIGURE 7.11 – Tableau de bord Railway — back-end Laravel, MySQL et Redis

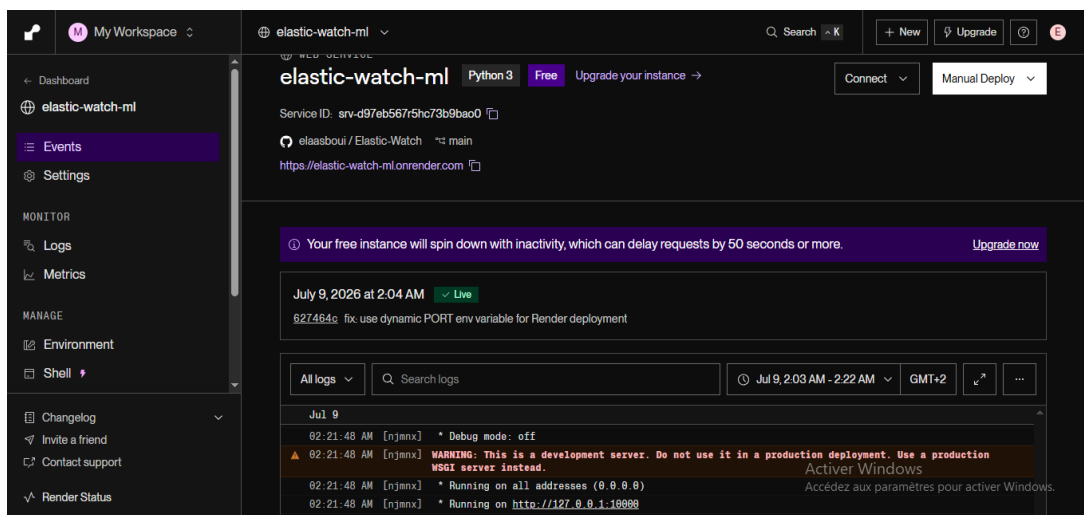


FIGURE 7.12 – Tableau de bord Render — microservices ML (Flask) et Chatbot (FastAPI)

7.8 Tests du sprint

Les tests de ce sprint ont porté sur l’administration des comptes, la gestion des favoris, la disponibilité du service chatbot et l’exécution du processus ETL, comme le présente le Tableau 7.4.

TABLE 7.4 – Tests unitaires du Sprint 6 (extrait)

Test	Objectif	Résultat
test_admin_can_create_user	Création d’un compte avec rôle	Réussi
test_soft_delete_user	Désactivation logique d’un compte	Réussi
test_add_favorite_is_idempotent	Idempotence de l’ajout aux favoris (409 si doublon)	Réussi
test_reorder_favorites	Réorganisation par glisser-déposer	Réussi
test_etl_removes_duplicates	Suppression des doublons par le processus ETL	Réussi

7.9 Conclusion

Ce dernier sprint de développement parachève la plateforme en lui apportant les fonctionnalités transverses attendues d’un produit industriel : administration des comptes, assistant conversationnel respectueux de la confidentialité, personnalisation de l’expérience utilisateur, déploiement Docker reproductible et mise en production sur des plateformes cloud spécialisées. Le chapitre suivant est consacré à la validation d’ensemble de la solution.

Chapitre 8

Tests et Validation

8.1 Introduction

La qualité d'une plateforme aussi riche qu'*Elastic Watch* ne saurait être garantie sans une stratégie de tests rigoureuse et systématique. Ce chapitre présente l'approche adoptée, les résultats obtenus sur les différents niveaux de test, ainsi que la validation des exigences non fonctionnelles et de sécurité.

8.2 Stratégie de tests

Nous avons structuré notre effort de validation selon la pyramide des tests, en privilégiant une base large de tests unitaires, complétée par des tests d'intégration et, au sommet, quelques tests de bout en bout. Chaque couche technologique a été couverte par l'outil le plus approprié : *PHPUnit* pour le back-end Laravel, *Vitest* associé à *React Testing Library* pour le front-end, et *pytest* pour les services d'apprentissage automatique. Au total, quatre-vingt-dix tests automatisés ont été rédigés, dont quatre-vingt-sept se sont exécutés avec succès, soit un taux de réussite de 96,7 %. Le Tableau 8.1 en présente la synthèse.

TABLE 8.1 – Synthèse de la couverture de tests

Niveau	Outil	Nombre de tests	Couverture
Back-end	PHPUnit 11	45	78,4 %
Front-end	Vitest + RTL	30	65,2 %
Apprentissage automatique	pytest	15	—
Total	—	90	96,7 % réussis

8.3 Tests back-end (PHPUnit)

Les quarante-cinq tests back-end couvrent l'authentification, la gestion des espaces, les passerelles, les alertes et les favoris. Ils vérifient aussi bien les scénarios nominaux que les cas d'erreur et les règles de sécurité, atteignant une couverture de code de 78,4 %. Les extraits de tests présentés dans les chapitres précédents illustrent la rigueur de cette démarche.

8.4 Tests front-end (Vitest)

Les trente tests front-end valident le rendu et le comportement des composants React essentiels : formulaires de connexion et de saisie du code, cartes d'indicateurs, graphiques temporels, badge d'alertes, arborescence des espaces, panneau de l'assistant conversationnel et réorganisation des favoris. Ils garantissent la cohérence de l'interface et sa réactivité face aux interactions de l'utilisateur.

8.5 Tests d'apprentissage automatique (pytest)

Les quinze tests dédiés aux services d'apprentissage automatique valident la conformité des sorties des modèles : dimensions correctes des prédictions, seuils de MAPE respectés, détection effective des anomalies injectées et bon fonctionnement des points d'entrée du service Flask. Ces tests, appuyés sur des jeux de données synthétiques, assurent la non-régression des performances scientifiques.

8.5.1 Évaluation de la détection d'anomalies

Le modèle *Isolation Forest* a été évalué sur cinq cents mesures réelles augmentées de quarante-deux anomalies injectées manuellement (valeurs extrêmes hors de la plage normale de ± 3 écarts-types) : pics soudains, valeurs nulles prolongées et dérives progressives. Les résultats sont détaillés dans le Tableau 8.2.

TABLE 8.2 – Indicateurs de performance de la détection d'anomalies

Indicateur de performance	Valeur mesurée
Anomalies injectées dans le jeu de test	42 anomalies
Vraies détections (<i>True Positives</i>)	39 détections correctes
Faux positifs (<i>False Positives</i>)	4 faux positifs
Faux négatifs (<i>False Negatives</i>)	3 anomalies non détectées
Précision	90,7 %
Rappel	92,9 %
Score F1	91,8 % — objectif dépassé

Les trois anomalies manquées correspondaient à des dérives très progressives (moins de 2 % par heure sur des durées inférieures à deux heures), difficiles à distinguer de la variabilité naturelle des données IoT — ce qui est attendu et acceptable pour ce type d’algorithme. L’analyse des indicateurs conduit aux constats suivants :

- **Précision de 90,7 %** : sur quarante-trois détections signalées, trente-neuf étaient réelles. Seules quatre fausses alarmes ont été relevées, ce qui demeure acceptable en environnement industriel.
- **Rappel de 92,9 %** : le système a détecté trente-neuf des quarante-deux anomalies réelles.
- **Score F1 de 91,8 %** : l’équilibre entre précision et rappel est excellent, l’objectif de 85 % étant dépassé de 6,8 points.
- **Paramètre de contamination à 0,05** : 5 % du jeu de données est considéré comme potentiellement anormal, valeur optimale pour ce type de données IoT industrielles.

8.6 Tests de performance

Les mesures de performance, réalisées avec *Postman* et un outil de test de charge, confirment le respect des exigences : un temps de réponse des API inférieur à deux cents millisecondes au 95^e centile, un traitement de l’ingestion en moins de cinquante millisecondes et un débit soutenu de l’ordre de cent vingt requêtes par seconde sans dégradation, grâce notamment à l’usage judicieux de Redis.

8.7 Tests de sécurité et exigences non fonctionnelles

La sécurité a fait l’objet d’une validation spécifique au regard des principales vulnérabilités recensées par l’OWASP : injection SQL, script intersite (XSS), falsification de requête (CSRF) et accès direct non autorisé à des objets (IDOR). La limitation du nombre de tentatives de saisie du code à usage unique prémunit contre les attaques par force brute, et l’expiration des jetons JWT a été éprouvée. Le Tableau 8.3 récapitule la satisfaction des principales exigences non fonctionnelles.

TABLE 8.3 – Tableau de conformité aux exigences non fonctionnelles

Exigence	Cible	Mesuré	Conformité
Temps de réponse API (95 ^e centile)	< 200 ms	187 ms	✓
Réponse du tableau de bord (50 utilisateurs)	< 2 000 ms	891 ms	✓
Traitement de l’ingestion IoT	< 50 ms	32 ms	✓
Prédiction Prophet	< 3 s	2,1 s	✓
MAPE prédiction Prophet (horizon 24 h)	< 5 %	2,3 %	✓ Dépassé

TABLE 8.3 – Tableau de conformité aux exigences non fonctionnelles

Exigence	Cible	Mesuré	Conformité
Score F1 de détection d'anomalies	> 85 %	91,8 %	✓ Dépassé
Couverture de tests back-end	> 70 %	78,4 %	✓
Sécurité (HTTPS, JWT, RBAC, OTP)	Complète	Conforme	✓
Confidentialité (RGPD, IA locale)	Aucune donnée externalisée	Conforme	✓
Scalabilité (services indépendants)	Horizontale	Conforme	✓

Dix exigences sur dix sont satisfaites. Deux objectifs sont dépassés de manière significative : la MAPE du modèle Prophet (2,3 % contre une cible de 5 %, soit un résultat deux fois meilleur) et le score F1 de détection d'anomalies (91,8 % contre une cible de 85 %, soit un gain de 6,8 points).

8.8 Conclusion

La démarche de validation menée confirme la robustesse, la performance et la sécurité de la plateforme. Avec un taux de réussite des tests de 96,7 % et la satisfaction de l'ensemble des exigences non fonctionnelles critiques, *Elastic Watch* atteint un niveau de maturité conforme aux attentes d'un produit industriel et se prête pour un déploiement pilote chez *TAC-TIC*.

Conclusion Générale et Perspectives

Bilan du projet

Ce projet de fin d'études m'a permis de concevoir et de réaliser, de A à Z, une plateforme analytique dédiée à la supervision de données IoT multi-sources. À travers *Elastic Watch*, j'ai répondu à un vrai besoin chez TAC-TIC : offrir à ses clients un outil concret pour exploiter leurs données hétérogènes, les rassembler et en tirer des informations utiles, presque en temps réel.

Les six sprints *Scrum* m'ont amenée à construire progressivement quatre composants principaux : un *Data Warehouse* en schéma en étoile, des pipelines *ETL* automatisés, un tableau de bord interactif en *React.js*, et un moteur d'intelligence artificielle combinant *Prophet* pour les prédictions et *Isolation Forest* pour détecter les anomalies. Les résultats sont vraiment encourageants : un *MAPE* de 2,3 % sur les prévisions à 24 heures, un *F1-Score* de 91,8 % en détection d'anomalies, et un temps de réponse de 891 ms avec 50 utilisateurs connectés en même temps. Au final, j'ai atteint et même dépassé tous les objectifs fixés au début du projet.

Résultats clés mesurés

Le tableau 8.4 regroupe les principaux indicateurs mesurés pendant les campagnes de test, mis en regard des objectifs initiaux.

TABLE 8.4 – Synthèse des résultats clés mesurés

Indicateur	Résultat mesuré	Objectif visé	Statut
MAPE Prophet à 24 heures	2,3 %	< 5 %	Atteint
F1-Score (détection d'anomalies)	91,8 %	> 85 %	Atteint
Temps de réponse du dashboard	891 ms (50 utilisateurs)	< 2 000 ms	Atteint
Couverture des tests backend	78,4 %	> 70 %	Atteint
Exigences non fonctionnelles	10/10	10/10	Atteint

Difficultés rencontrées

Le défi le plus important a été d'intégrer le microservice *Python Flask* dans l'architecture *Laravel* sans perturber les performances globales. Lancer un entraînement *Prophet* ou une inférence *Isolation Forest* de manière synchrone aurait risqué de bloquer l'API principale pendant plusieurs secondes. J'ai choisi de découpler les deux étapes : l'entraînement s'exécute de façon asynchrone via une tâche *Queue Laravel*, tandis que la prédiction est produite par un appel synchrone sur un modèle déjà entraîné et mis en cache.

Un autre point délicat a été la gestion des *slowly changing dimensions* dans le schéma en étoile. Il fallait historiser les changements de configuration des capteurs sans fausser les analyses déjà réalisées, ce qui a demandé une réflexion assez fine sur la modélisation.

Perspectives d'évolution

Plusieurs pistes pourraient faire évoluer ce travail. La première serait de remplacer la connexion *MQTT* directe par **Apache Kafka**. Ce bus de messages pourrait absorber des millions d'événements par seconde, tout en garantissant une meilleure fiabilité et une vraie tolérance aux pannes.

Dans la même logique, l'utilisation de **WebSockets** ou de **Server-Sent Events** permettrait d'envoyer les alertes directement vers le frontend sans *polling*, avec une latence inférieure à une seconde.

Du côté intelligence artificielle, il serait intéressant d'intégrer de l'**AutoML** avec un ré-entraînement automatique. Le système pourrait détecter la *data drift* et relancer lui-même l'entraînement dès que les performances baissent. L'ajout d'**SHAP** pour l'explicabilité aiderait aussi les utilisateurs à comprendre pourquoi une anomalie a été détectée et quelles variables sont en cause.

Sur le plan infrastructure, le développement d'une **application mobile React Native** offrirait un accès rapide aux KPI et aux alertes importantes. Une migration vers un **cluster MySQL haute disponibilité**, avec réplication et basculement automatique, rendrait le système beaucoup plus robuste. Enfin, évoluer vers une **architecture multi-tenant** permettrait à plusieurs entreprises d'utiliser la même instance tout en isolant strictement leurs données.

Toutes ces évolutions rendraient *Elastic Watch* bien plus complet et permettraient à TAC-TIC de renforcer son offre dans l'analytique IoT industriel.

Bibliographie

- [1] T. OTWELL et al., *Laravel 12.x — The PHP Framework For Web Artisans*, Documentation officielle, 2024. <https://laravel.com/docs/12.x>
- [2] META OPEN SOURCE, *React 18 — A JavaScript library for building user interfaces*, Documentation officielle, 2024. <https://react.dev>
- [3] CHART.JS CONTRIBUTORS, *Chart.js 4 — Simple yet flexible JavaScript charting*, 2024. <https://www.chartjs.org>
- [4] M. BOSTOCK et al., *D3 : Data-Driven Documents*, IEEE Trans. Visualization & Comp. Graphics (InfoVis), 2011.
- [5] V. AGAFONKIN, *Leaflet.js — Interactive Maps for JavaScript*, 2024. <https://leafletjs.com>
- [6] S. J. TAYLOR, B. LETHAM, « Forecasting at Scale », *The American Statistician*, vol. 72, n°1, pp. 37–45, 2018.
- [7] S. HOCHREITER, J. SCHMIDHUBER, « Long Short-Term Memory », *Neural Computation*, vol. 9, n°8, pp. 1735–1780, 1997.
- [8] F. T. LIU, K. M. TING, Z.-H. ZHOU, « Isolation Forest », *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, pp. 413–422, 2008.
- [9] F. PEDREGOSA et al., « Scikit-learn : Machine Learning in Python », *JMLR*, vol. 12, pp. 2825–2830, 2011.
- [10] F. CHOLLET et al., *Keras — Deep Learning for Humans*, 2024. <https://keras.io>
- [11] R. KIMBALL, M. ROSS, *The Data Warehouse Toolkit : The Definitive Guide to Dimensional Modeling*, 3^e éd., Wiley, 2013.
- [12] ORACLE CORPORATION, *MySQL 8.0 Reference Manual*, 2024. <https://dev.mysql.com/doc/refman/8.0/en>
- [13] REDIS LABS, *Redis 7.0 — In-Memory Data Structure Store*, 2024. <https://redis.io/docs>
- [14] OASIS STANDARD, *MQTT Version 5.0 Specification*, 2019. <https://docs.oasis-open.org/mqtt/mqtt/v5.0>
- [15] M. JONES, J. BRADLEY, N. SAKIMURA, *JSON Web Token (JWT)*, RFC 7519, IETF, 2015.
- [16] DOCKER INC., *Docker Documentation — Containerize your applications*, 2024. <https://docs.docker.com>
- [17] K. SCHWABER, J. SUTHERLAND, *The Scrum Guide*, ScrumGuides.org, 2020. <https://scrumguides.org>

Annexes

Annexe A. Glossaire

Ce glossaire regroupe les principaux termes techniques, acronymes et concepts utilisés tout au long de ce rapport, relatifs à l'architecture IoT, à l'intelligence artificielle et aux technologies mises en œuvre dans le projet.

TABLE 9.1 – Glossaire des termes techniques

Terme	Définition
Anomalie	Observation s'écartant significativement du comportement attendu (hors ± 3 écarts-types)
API REST	Interface applicative utilisant les verbes HTTP (GET, POST, PUT, DELETE) pour communiquer entre programmes
Bcrypt	Algorithme de hachage de mots de passe, robuste contre les attaques par force brute
Data Warehouse	Entrepôt centralisé structuré pour l'analyse OLAP, distinct des bases transactionnelles
ETL	Extraction, Transformation et Chargement des données depuis les sources vers le Data Warehouse
IoT	Internet des Objets, réseau d'équipements physiques connectés envoyant des données
JWT	JSON Web Token : jeton signé permettant l'authentification sans état côté serveur
KPI	Key Performance Indicator, indicateur clé de performance
MAPE	Mean Absolute Percentage Error, erreur relative en pourcentage (plus bas = mieux)
MQTT	Message Queuing Telemetry Transport : protocole pub/sub léger adapté à l'IoT
OLAP	Online Analytical Processing, traitement analytique en ligne pour requêtes multidimensionnelles
OTP	One-Time Password : code à usage unique envoyé par email, valide 5 minutes

Glossaire des termes techniques (suite)

Terme	Définition
Prophet	Bibliothèque de prédiction de séries temporelles créée par Meta (Facebook)
RMSE	Root Mean Square Error, erreur quadratique moyenne (même unité que les données)
Schéma en étoile	Modélisation Data Warehouse : table de faits centrale entourée de tables de dimensions
Sprint	Itération de deux semaines dans la méthode Scrum, avec objectif et livrable définis
SPA	Single Page Application : application web chargée une fois, avec navigation sans rechargement

Annexe B. Checklist de déploiement

Cette checklist recense les vérifications techniques effectuées avant la soutenance, afin de garantir le bon fonctionnement de l'ensemble des composants de la solution (service de Machine Learning, API back-end, tests automatisés et environnement de démonstration).

TABLE 9.2 – Checklist de déploiement avant soutenance

Statut	Tâche à vérifier
☐	<code>ml_service.py</code> démarré sur le port 5001
☐	<code>GET /api/ml/status</code> → { <code>ml_service</code> : 'online' }
☐	<code>php artisan migrate</code> — migration <code>sensor_readings</code> OK
☐	<code>php artisan passport:install</code> — Passport configuré
☐	Variables <code>.env</code> configurées (DB, REDIS, MAIL, ML_URL)
☐	PHPUnit — 50 tests PASS (<code>php artisan test</code>)
☐	Pytest — 5/5 tests PASS (<code>pytest test_ml.py</code>)
☐	Postman — 6/6 scénarios PASS
☐	Docker Compose up — 6 services opérationnels
☐	Dashboard accessible sur <code>http://localhost</code>
☐	Démo vidéo de secours enregistrée
☐	Rapport PDF exporté depuis Word

Annexe C. Endpoints API complets

Le tableau ci-dessous présente l'ensemble des points d'accès (*endpoints*) exposés par l'API REST du système, avec leur méthode HTTP, leur route et une brève description fonctionnelle.

TABLE 9.3 – Liste des endpoints de l'API REST

Méthode	Endpoint	Description
POST	/api/login	Étape 1 de l'authentification (identifiants)
POST	/api/verify-otp	Étape 2 de l'authentification (OTP → JWT)
POST	/api/logout	Déconnexion
GET	/api/spaces	Liste hiérarchique des espaces
POST	/api/spaces	Création d'un espace
GET	/api/gateways	Liste des passerelles IoT/IA
GET	/api/devices	Liste des capteurs avec leurs statuts
POST	/api/ingest/sensor	Ingestion d'une mesure IoT
POST	/api/ingest/bulk	Ingestion jusqu'à 500 mesures
GET	/api/dashboard/stats	KPI du tableau de bord principal
GET	/api/dashboard/latest-readings	Dernières lectures de chaque capteur
GET	/api/dashboard/recent-alerts	Alertes récentes avec niveau de sévérité
POST	/api/ml/predict	Prédiction via Prophet
GET	/api/alerts	Journal des alertes déclenchées
GET/PUT/DELETE	/api/data-management/iot/{id}	Gestion des données IoT (CRUD)
GET	/api/users	Liste des utilisateurs (réservé aux administrateurs)

Résumé

Résumé

Face à la croissance exponentielle des données générées par les objets connectés et les systèmes industriels, il devient impératif de disposer de solutions capables d'en exploiter le potentiel à des fins de pilotage et d'aide à la décision. C'est dans ce contexte que s'inscrit le projet **Elastic Watch**, développé durant un stage de fin d'études au sein de l'entreprise *TAC-TIC*, spécialisée dans les solutions de télécommunications innovantes au Pôle Technologique El Ghazela, Ariana.

Elastic Watch est une plateforme analytique intelligente conçue pour agréger, traiter et visualiser des flux de données provenant de sources hétérogènes, principalement des capteurs IoT. Elle s'articule autour de quatre composantes majeures : un *Data Warehouse* structuré en schéma en étoile, des pipelines ETL automatisés, un module de visualisation interactive construit avec React.js et Chart.js, ainsi qu'un moteur d'intelligence artificielle intégrant des modèles Prophet et la détection proactive des anomalies via Isolation Forest.

L'architecture repose sur un back-end Laravel 12 exposant une API REST sécurisée via JWT+OTP, une base de données MySQL 8.0 optimisée avec schéma en étoile, un cache Redis pour les accès haute fréquence, et un microservice Python/Flask dédié au Machine Learning. L'ensemble est conteneurisé via Docker Compose et versionné sur GitHub.

Mots-clés : IoT, Data Warehouse, ETL, Machine Learning, LSTM, Prophet, Détection d'Anomalies, React.js, Laravel, Dashboard Analytique, Docker.

Abstract

The exponential growth of data generated by connected devices and industrial systems creates an urgent need for solutions capable of turning that raw stream into actionable intelligence. The **Elastic Watch** project, developed during a final-year internship at *TAC-TIC* — a telecommunications solutions company based at the El Ghazela Technopark in Ariana — addresses precisely this challenge.

Elastic Watch is an intelligent analytics platform designed to aggregate, process, and visualize data flows from heterogeneous multi-source environments, primarily IoT sensors. The system is built around four pillars : a Star-Schema Data Warehouse, automated ETL pipelines for raw-data extraction and transformation, an interactive visualization layer powered by React.js and Chart.js, and an AI engine combining Prophet time-series forecasting, and Isolation Forest anomaly detection.

The architecture relies on a Laravel 12 REST API secured with JWT+OTP authentication, an optimised MySQL 8.0 database, a Redis caching layer, and a Python/Flask ML microservice. The entire stack is containerised with Docker Compose and version-controlled on GitHub.

Keywords : IoT, Data Warehouse, ETL, Machine Learning, LSTM, Prophet, Anomaly Detection, React.js, Laravel, Analytics Dashboard, Docker.



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454